

Texas College of Management & IT
**Bachelor of Computer Science (Hons) (Network Technology
& Cybersecurity)**

**Project Title: Practical Implementation of a Database-
Driven Student Information Management System**

Submitted By:

Name: Praya Pokharel

LCID: LC00017003233

Year/Semester: 2nd Year / 4th Semester

Section A

Submitted To:

Lecturer Name: Dr. Sushil Bhattarai

Course: Database Management Systems(DMS - 236)

Date of Submission: 27th July, 2026

TABLE OF CONTENTS

TABLE OF CONTENTS	I
1. LIST OF TABLES	III
2. LIST OF FIGURES	III
3. ABBREVIATIONS AND ACROYNOMS	V
4. INTRODUCTION	VI
4.1 Database Management System	VI
4.2 Student Information Management System	VI
4.3 Objectives	VI
4.4 Scope	VII
4.5 Technologies Used	VII
5. PRACTICAL 1 : CONCEPTUAL & LOGICAL DESIGN	8
5.1 Entities Identified	8
5.2 Entity–Relationship Diagram (Chen Notation)	8
5.3 Normalization	9
6. PRACTICAL 2 : DATABASE CREATION	20
6.1 Database Creation	20
6.2 Table Creation	21
6.3 Sample Data Insertion	29
7. PRACTICAL 3: CRUD OPERATIONS & QUERIES	35
7.1 INSERT	35
7.2 UPDATE	35
7.3 DELETE	36
7.4 SELECT with JOIN	36
7.5 Search Conditions	37
7.6 ORDER BY, DISTINCT, TOP, and Aliases	38
8. PRACTICAL 4: RELATIONAL ALGEBRA & TRANSACTIONS	41
8.1 Relational Algebra Operators	41
8.2 Transaction Management	43
9. PRACTICAL 5: DATABASE OPERATIONS	45
9.1 SQL Joins	45
9.2 Aggregate Functions	48

9.3 Grouping & Filtering	49
9.4 Subquery	50
9.5 View	51
9.6 Stored Procedure	51
9.7 Trigger	52
9.8 Index	53
10. CONCLUSION	54
11. REFERENCES	54
12. APPENDIX	55

1. LIST OF TABLES

Table 1 : Unnormalized form of data	10
Table 2 : data arranged in first normal form (1NF)	12
Table 3 : Second normal form data for student	13
Table 4 : Second normal form data for course	13
Table 5 : Second normal form data for enrollment	14
Table 6 : Third Normal form data for department table	15
Table 7 : Third Normal form data for Admin table	15
Table 8 :Third Normal form data for Faculty table	15
Table 9 : Third Normal form data for Student table	17
Table 10 : Third Normal form data for Courses table	18
Table 11 : Third Normal form data for enrollment table	18
Table 12 : Third Normal form data for feedback table	20

2. LIST OF FIGURES

Figure 1 : Entity-Relationship Diagram (Chen Notation) of the Student Information Management System	8
Figure 2 : Creating database	21
Figure 3 : Creation of the Department table.	22
Figure 4 : Creation of the Admin table in Microsoft SQL Server.	23
Figure 5 : Creation of the Students table.	24
Figure 6 : Creation of the Faculty table.	25
Figure 7 : Creation of the Courses table.	26
Figure 8 : Creation of the Enrollment table.	27
Figure 9 : Creation of the Feedback table.	29
Figure 10 : Inserting sample records into the Department table.	29
Figure 11 : Inserting sample records into the Admin table.	30
Figure 12 : Inserting sample records into the Faculty table.	30
Figure 13 : Inserting sample records into the Courses table.	31
Figure 14 : Inserting sample records into the Students table.	31
Figure 15 : Inserting sample records into the Enrollment table.	32
Figure 16 : Inserting sample records into the Feedback table.	32

Figure 17 : verification of values in department, faculty and course table	33
Figure 18 : verification of value insertion in student table	34
Figure 19 : value insertion verification in enrollment table	34
Figure 20 : values inserion verification in feedback table	34
Figure 21 : verification of inserted recrd in admin table	34
Figure 22 Inserting new records into the Students and Courses tables.	35
Figure 23 : Updating existing records in the Students and Courses tables.	36
Figure 24 : Deleting a selected enrollment record from the Enrollment table and recently inserted courses	36
Figure 25 : Retrieving student details with enrolled courses using INNER JOIN.	37
Figure 26 : Executing search queries using the LIKE, BETWEEN, and IN operators.	38
Figure 27 : Executing SQL queries using ORDER BY	39
Figure 28 : Executing SQL queries using DISTINCT	39
Figure 29 : Executing SQL queries using TOP	40
Figure 30 : Executing SQL queries using Aliases.	40
Figure 31 : Execution of relational algebra operations that include selection and projection	41
Figure 32 : Execution of relational algebra operations that include join	42
Figure 33 : Execution of relational algebra operations that include union	42
Figure 34 : Execution of relational algebra operations that include difference and Cartesian product	43
Figure 35 : Execution of transaction management operations using COMMIT, ROLLBACK, and SAVEPOINT.	44
Figure 36 : Verification of transaction management operations.	44
Figure 37 : Executing SQL join operations using inner join	45
Figure 38 : Executing SQL join operation using left join	46
Figure 39 : Executing SQL join operation using right join	46
Figure 40 : Executing SQL join operation using self join	46
Figure 41 : Executing SQL join operation using full join	47
Figure 42 : Executing SQL join operation using cross join	47
Figure 43 : Executing aggregate function queries using COUNT	48
Figure 44 : Executing aggregate function queries using SUM	48
Figure 45 : Executing aggregate function queries using AVG	49

Figure 46 : Executing aggregate function queries using MAX and MIN	49
Figure 47 : Executing GROUP BY and HAVING queries.	50
Figure 48 : Executing a sub-query to retrieve student records.	50
Figure 49 : Creating and retrieving data using a SQL view	51
Figure 50 : Creating and executing a stored procedure.	52

3. ABBREVIATIONS AND ACROYNOMS

- DBMS – Database Management System
- RDBMS – Relational Database Management System
- SIMS – Student Information Management System
- SQL – Structured Query Language
- T-SQL – Transact-SQL
- SSMS – SQL Server Management Studio
- UNF – Unnormalised Form
- 1NF / 2NF / 3NF – First / Second / Third Normal Form
- CRUD – Create, Read, Update, Delete
- PK – Primary Key
- FK – Foreign Key
- HOD – Head of Department
- ACID – Atomicity, Consistency, Isolation, Durability
- GUI – Graphical User Interface
- ER Diagram – Entity-Relationship Diagram

4. INTRODUCTION

4.1 Database Management System

In simple terms, a DBMS (Database Management System) is a software system that allows one to store data in an organized manner rather than scattering it across files haphazardly. This system will ensure that the data remains accurate by use of constraints/keys, supports multi-user transactions and enables querying using SQL. SQL Server, used in this project, is an RDBMS based on the relational model of Codd.

4.2 Student Information Management System

A Student Information Management System (SIMS) simply means having all those disparate elements such as information about students, courses, staff, enrollments, grading, feedback, and whatnot, which would typically be scattered in spreadsheets and on paper, entered into one cohesive database. The benefits include less duplication of efforts, maintaining consistency (such as not being able to enroll a student who does not even exist), and running meaningful queries.

4.3 Objectives

- ✧ Create a normalized database up to 3NF for students.
- ✧ Create this database in SQL Server.
- ✧ Perform CRUD operations using SQL.
- ✧ Relational algebra should be shown using equivalent SQL statements.
- ✧ Transactions should use COMMIT, ROLLBACK and SAVEPOINT commands.
- ✧ Also include other features like views, stored procedures and triggers.

4.4 Scope

- ✧ It contains five key entities such as Department, Student, Faculty, Course, and Admin (for system control).
- ✧ Enrollment and Feedback serve as associative entities between these.
- ✧ There is no GUI; all is presented directly through the database level via T-SQL language.
- ✧ This is sufficient to achieve all objectives specified in DMS-236.

4.5 Technologies Used

- ✧ SQL Server 2022 (Developer Edition) — the database engine
- ✧ SSMS — for writing and running queries
- ✧ T-SQL — used for schema, queries, views, procedures, triggers
- ✧ MS Word — for writing up the report

5. PRACTICAL 1 : CONCEPTUAL & LOGICAL DESIGN

5.1 Entities Identified

The seven entities considered for the given project scenario are Department, Admin, Students, Faculty, Courses, Enrollment, and Feedback. The Enrollment and Feedback are junction entities which help to solve many-to-many relationships among Students, Courses, and Faculty. Admin is an independent entity with two relationships associated with it, namely student registration and faculty approval.

5.2 Entity–Relationship Diagram (Chen Notation)

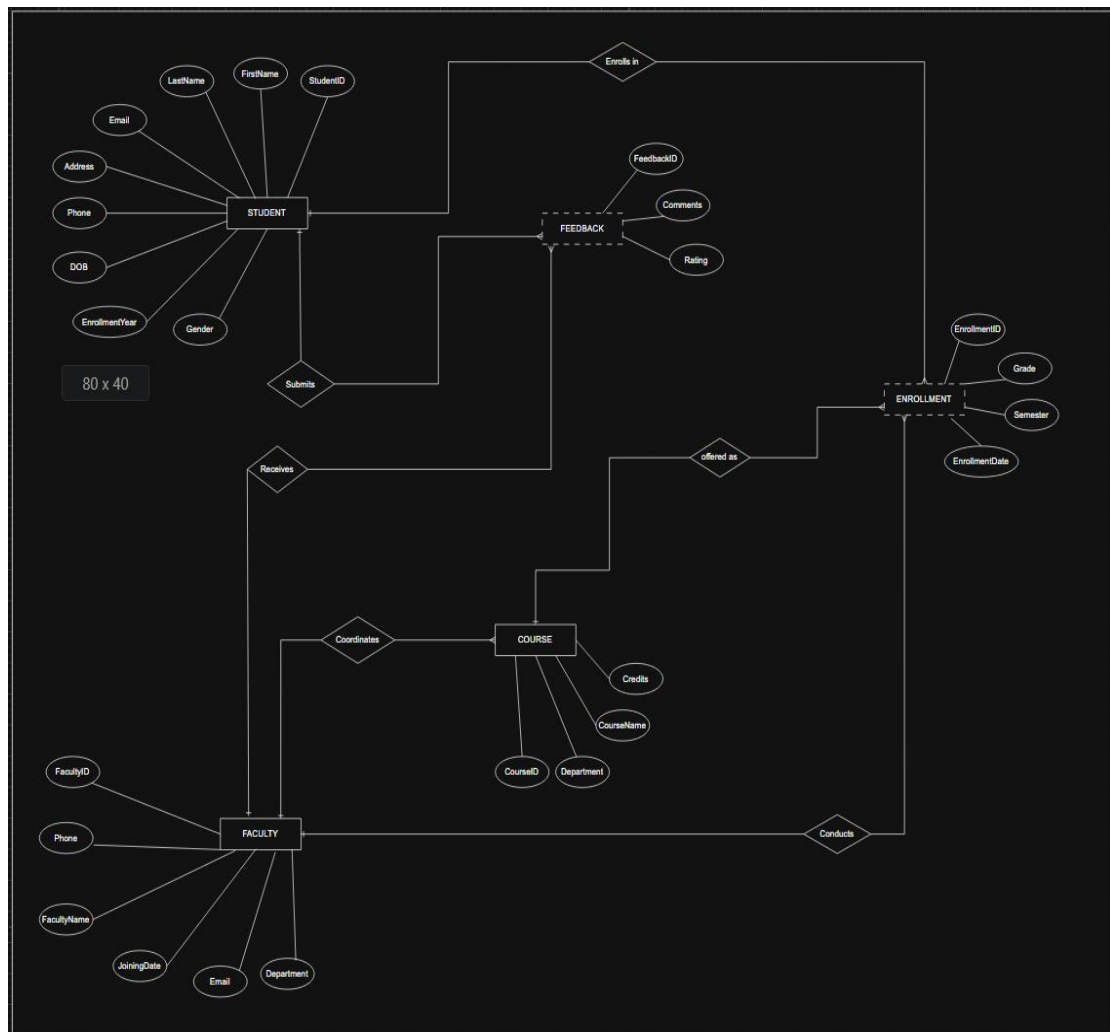


Figure 1: Entity-Relationship Diagram (Chen Notation) of the Student Information Management System

5.3 Normalization

➤ Unnormalized Form (UNF)

The Unnormalized Form (UNF) represents the initial data structure where all student, course, faculty, department, enrollment, and feedback information is stored in a single table. This structure contains repeating groups and redundant data, making it difficult to maintain data consistency and increasing the risk of update, insertion, and deletion anomalies.

StudentID	StudentName	DepartmentName	RegisteredBy (Admin)
1	Ram Bahadur	Computer Science	Suman Shrestha
2	Sita Gurung	Computer Science	Suman Shrestha
3	Hari Shrestha	Mathematics	Nisha Karki
4	Gita Rai	Cybersecurity	Nisha Karki
5	Suresh Karki	Computer Science	Suman Shrestha
6	Anita Magar	Cybersecurity	Nisha Karki
7	Bikash Rai	Computer Science	Suman Shrestha
8	Kabita Thapa	Business Studies	Nisha Karki
9	Prakash Adhikari	Computer Science	Suman Shrestha
10	Manisha Poudel	Mathematics	Suman Shrestha
11	Nabin Tamang	Cybersecurity	Nisha Karki
12	Sabina Lama	Computer Science	Suman Shrestha
13	Rajesh KC	Business Studies	Nisha Karki
14	Puja Basnet	Mathematics	Suman Shrestha
15	Deepak Khadka	Cybersecurity	Nisha Karki

Enrollments {Course Semester Grade}	Feedback {Faculty Rating Comment}
{ Database Management Systems Fall 2024 A , Web Technologies Spring 2025 B+ }	{ Dr. Anil Sharma 5 "Excellent teaching" , Dr. Anil Sharma 4 "Good pace" }
{ Data Structures Fall 2024 B+ , Database Management Systems Spring	{ Ms. Sunita Lama 4 "Very helpful" , Dr. Anil Sharma 5 "Clear

2025 A- }	explanations" }
{ Calculus I Fall 2024 A- , Network Security Spring 2025 B }	{ Dr. Rita Koirala 5 "Explains clearly" , Prof. Bikash Thapa 4 "Practical examples" }
{ Network Security Fall 2024 A , Ethical Hacking Spring 2025 A- }	{ Prof. Bikash Thapa 3 "Needs more examples" , Dr. Kabita Nepal 4 "Engaging labs" }
{ Database Management Systems Fall 2024 B , Operating Systems Spring 2025 B+ }	{ Dr. Anil Sharma 4 "Good instructor" , Ms. Sunita Lama 3 "Fast paced" }
{ Ethical Hacking Fall 2024 A , Network Security Spring 2025 A }	{ Dr. Kabita Nepal 5 "Best faculty" , Prof. Bikash Thapa 4 "Well organised" }
{ Database Management Systems Fall 2024 A- , Operating Systems Spring 2025 B }	{ Dr. Anil Sharma 4 "Solid fundamentals" , Ms. Sunita Lama 3 "Could slow down" }
{ Business Communication Fall 2024 B+ }	{ Mr. Deepak Basnet 4 "Friendly approach" }
{ Web Technologies Fall 2024 A , Data Structures Spring 2025 B }	{ Dr. Anil Sharma 5 "Very inspiring" , Ms. Sunita Lama 3 "Average pace" }
{ Calculus I Fall 2024 B }	{ Dr. Rita Koirala 4 "Detailed notes" }
{ Ethical Hacking Fall 2024 A- }	{ Dr. Kabita Nepal 5 "Great mentor" }
{ Data Structures Fall 2024 B+ }	{ Ms. Sunita Lama 4 "Helpful feedback" }
{ }	{ }
{ }	{ }
{ }	{ }

Table 1: Unnormalized form of data

➤ First Normal Form (1NF)

The First Normal Form (1NF) eliminates repeating groups by ensuring that each column contains only atomic (single) values and each record is uniquely identified.

Although the data is more organized, some redundancy and partial dependencies may still exist.

StudentID	StudentName	DepartmentName	RegisteredBy (Admin)
1	Ram Bahadur	Computer Science	Suman Shrestha
1	Ram Bahadur	Computer Science	Suman Shrestha
2	Sita Gurung	Computer Science	Suman Shrestha
2	Sita Gurung	Computer Science	Suman Shrestha
3	Hari Shrestha	Mathematics	Nisha Karki
3	Hari Shrestha	Mathematics	Nisha Karki
a4	Gita Rai	Cybersecurity	Nisha Karki
4	Gita Rai	Cybersecurity	Nisha Karki
5	Suresh Karki	Computer Science	Suman Shrestha
5	Suresh Karki	Computer Science	Suman Shrestha
6	Anita Magar	Cybersecurity	Nisha Karki
6	Anita Magar	Cybersecurity	Nisha Karki
7	Bikash Rai	Computer Science	Suman Shrestha
7	Bikash Rai	Computer Science	Suman Shrestha
8	Kabita Thapa	Business Studies	Nisha Karki
9	Prakash Adhikari	Computer Science	Suman Shrestha
9	Prakash Adhikari	Computer Science	Suman Shrestha
10	Manisha Poudel	Mathematics	Suman Shrestha
11	Nabin Tamang	Cybersecurity	Nisha Karki
12	Sabina Lama	Computer Science	Suman Shrestha

Course ID	CourseName	Credits	FacultyName	Semester	Grade
1	Database Management Systems	4	Dr. Anil Sharma	Fall 2024	A
7	Web Technologies	3	Dr. Anil Sharma	Spring 2025	B+
2	Data Structures	4	Ms. Sunita Lama	Fall 2024	B+
1	Database Management Systems	4	Dr. Anil Sharma	Spring 2025	A-

3	Calculus I	3	Dr. Rita Koirala	Fall 2024	A-
4	Network Security	4	Prof. Bikash Thapa	Spring 2025	B
4	Network Security	4	Prof. Bikash Thapa	Fall 2024	A
5	Ethical Hacking	3	Dr. Kabita Nepal	Spring 2025	A-
1	Database Management Systems	4	Dr. Anil Sharma	Fall 2024	B
8	Operating Systems	4	Ms. Sunita Lama	Spring 2025	B+
5	Ethical Hacking	3	Dr. Kabita Nepal	Fall 2024	A
4	Network Security	4	Prof. Bikash Thapa	Spring 2025	A
1	Database Management Systems	4	Dr. Anil Sharma	Fall 2024	A-
8	Operating Systems	4	Ms. Sunita Lama	Spring 2025	B
6	Business Communication	2	Mr. Deepak Basnet	Fall 2024	B+
7	Web Technologies	3	Dr. Anil Sharma	Fall 2024	A
2	Data Structures	4	Ms. Sunita Lama	Spring 2025	B
3	Calculus I	3	Dr. Rita Koirala	Fall 2024	B
5	Ethical Hacking	3	Dr. Kabita Nepal	Fall 2024	A-
2	Data Structures	4	Ms. Sunita Lama	Fall 2024	B+

Table 2: data arranged in first normal form (1NF)

➤ Second Normal Form (2NF)

The Second Normal Form (2NF) removes partial dependencies by separating data into related tables. Attributes that depend only on part of a composite key are moved to separate tables, reducing redundancy and improving data organization.

Student — depends on StudentID alone (still holds text that belongs elsewhere)

StudentID	StudentName	DepartmentName	RegisteredBy (Admin)
-----------	-------------	----------------	----------------------

1	Ram Bahadur	Computer Science	Suman Shrestha
2	Sita Gurung	Computer Science	Suman Shrestha
3	Hari Shrestha	Mathematics	Nisha Karki
4	Gita Rai	Cybersecurity	Nisha Karki
5	Suresh Karki	Computer Science	Suman Shrestha
6	Anita Magar	Cybersecurity	Nisha Karki
7	Bikash Rai	Computer Science	Suman Shrestha
8	Kabita Thapa	Business Studies	Nisha Karki
9	Prakash Adhikari	Computer Science	Suman Shrestha
10	Manisha Poudel	Mathematics	Suman Shrestha
11	Nabin Tamang	Cybersecurity	Nisha Karki
12	Sabina Lama	Computer Science	Suman Shrestha
13	Rajesh KC	Business Studies	Nisha Karki
14	Puja Basnet	Mathematics	Suman Shrestha
15	Deepak Khadka	Cybersecurity	Nisha Karki

Table 3: Second normal form data for student

Course — depends on CourseID alone

CourseID	CourseName	Credits	FacultyName
1	Database Management Systems	4	Dr. Anil Sharma
2	Data Structures	4	Ms. Sunita Lama
3	Calculus I	3	Dr. Rita Koirala
4	Network Security	4	Prof. Bikash Thapa
5	Ethical Hacking	3	Dr. Kabita Nepal
6	Business Communication	2	Mr. Deepak Basnet
7	Web Technologies	3	Dr. Anil Sharma
8	Operating Systems	4	Ms. Sunita Lama

Table 4: Second normal form data for course

Enrollment — the connecting table (needs the full key)

StudentID	CourseID	Semester	Grade
1	1	Fall 2024	A
1	7	Spring 2025	B+
2	2	Fall 2024	B+
2	1	Spring 2025	A-
3	3	Fall 2024	A-
3	4	Spring 2025	B
4	4	Fall 2024	A
4	5	Spring 2025	A-
5	1	Fall 2024	B
5	8	Spring 2025	B+
6	5	Fall 2024	A
6	4	Spring 2025	A
7	1	Fall 2024	A-
7	8	Spring 2025	B
8	6	Fall 2024	B+
9	7	Fall 2024	A
9	2	Spring 2025	B
10	3	Fall 2024	B
11	5	Fall 2024	A-
12	2	Fall 2024	B+

Table 5: Second normal form data for enrollment

➤ Third Normal Form (3NF)

The Third Normal Form (3NF) removes transitive dependencies by ensuring that all non-key attributes depend only on the primary key. The database is divided into well-structured tables such as Department, Admin, Students, Faculty, Courses, Enrollment, and Feedback, resulting in improved data integrity, reduced redundancy, and easier maintenance.

Department

DepartmentID	DepartmentName	HOD
1	Computer Science	Dr. Anil Sharma
2	Mathematics	Dr. Rita Koirala
3	Cybersecurity	Prof. Bikash Thapa
4	Business Studies	Dr. Meena Adhikari

Table 6: Third Normal form data for department table

Admin

AdminID	AdminName	Email	Role
1	Suman Shrestha	suman.admin@college.edu	System Administrator
2	Nisha Karki	nisha.admin@college.edu	Academic Registrar

Table 7: Third Normal form data for Admin table

Faculty

Faculty ID	FacultyName	Email	Phone	DepartmentID	Admin ID	JoiningDate
1	Dr. Anil Sharma	anil.sharma@college.edu	9800000001	1	1	2006-01-09
2	Dr. Rita Koirala	rita.koirala@college.edu	9800000002	2	1	2004-09-23
3	Prof. Bikash Thapa	bikash.thapa@college.edu	9800000003	3	2	2005-06-29
4	Ms. Sunita Lama	sunita.lama@college.edu	9800000004	1	1	2012-03-15
5	Mr. Deepak Basnet	deepak.basnet@college.edu	9800000005	4	2	2015-08-01
6	Dr. Kabita Nepal	kabita.nepal@college.edu	9800000006	3	2	2010-11-20

Table 8: Third Normal form data for Faculty table

Students

Student ID	First Name	Last Name	Gender	DOB	Email	Phone	Address	Enroll Year	Department ID	Admin ID
1	Ram	Bahadur	M	2003-05-12	ram.bahadur@student.edu	9811111111	Kathmandu	2022	1	1
2	Sita	Gurung	F	2003-08-20	sita.gurung@student.edu	9822222222	Lalitpur	2022	1	1
3	Hari	Shrestha	M	2002-11-02	hari.shrestha@student.edu	9833333333	Bhaktapur	2021	2	2
4	Gita	Rai	F	2004-01-15	gita.raistudent.edu	9844444444	Kathmandu	2023	3	2
5	Suresh	Karki	M	2003-03-30	suresh.karki@student.edu	9855555555	Pokhara	2022	1	1
6	Anita	Magar	F	2004-06-18	anita.magar@student.edu	9866666666	Kathmandu	2023	3	2
7	Bikas	Rai	M	2004-02-20	bikash.raistudent.edu	9877777777	Kathmandu	2023	1	1
8	Kabita	Thapa	F	2003-09-09	kabita.thapa@student.edu	9888888888	Chitwan	2022	4	2
9	Prakash	Adhikari	M	2002-12-25	prakash.adhikari@student.edu	9899999999	Kathmandu	2021	1	1

					t.edu					
10	Manisha	Poudel	F	2004-04-04	manisha.poudel@student.t.edu	980111111	Lalitpur	2023	2	1
11	Nabin	Tamang	M	2003-07-07	nabin.tamang@student.edu	980222222	Bhaktapur	2022	3	2
12	Sabina	Lama	F	2004-10-10	sabina.lama@student.edu	980333333	Kathmandu	2023	1	1
13	Rajesh	KC	M	2002-05-05	rajesh.kc@student.edu	980444444	Pokhara	2021	4	2
14	Puja	Basnet	F	2003-01-01	puja.basnet@student.edu	980555555	Kathmandu	2022	2	1
15	Deepak	Khadka	M	2004-03-03	deepak.khadka@student.edu	980666666	Lalitpur	2023	3	2

Table 9: Third Normal form data for Student table

Courses

CourseID	CourseName	Credits	DepartmentID	FacultyID
1	Database Management Systems	4	1	1
2	Data Structures	4	1	4
3	Calculus I	3	2	2
4	Network Security	4	3	3
5	Ethical Hacking	3	3	6
6	Business Communication	2	4	5
7	Web Technologies	3	1	1

8	Operating Systems	4	1	4
---	-------------------	---	---	---

Table 10: Third Normal form data for Courses table

Enrollment

EnrollmentID	StudentID	CourseID	Semester	Grade
1	1	1	Fall 2024	A
2	1	7	Spring 2025	B+
3	2	2	Fall 2024	B+
4	2	1	Spring 2025	A-
5	3	3	Fall 2024	A-
6	3	4	Spring 2025	B
7	4	4	Fall 2024	A
8	4	5	Spring 2025	A-
9	5	1	Fall 2024	B
10	5	8	Spring 2025	B+
11	6	5	Fall 2024	A
12	6	4	Spring 2025	A
13	7	1	Fall 2024	A-
14	7	8	Spring 2025	B
15	8	6	Fall 2024	B+
16	9	7	Fall 2024	A
17	9	2	Spring 2025	B
18	10	3	Fall 2024	B
19	11	5	Fall 2024	A-
20	12	2	Fall 2024	B+

Table 11: Third Normal form data for enrollment table

Feedback

FeedbackID	StudentID	FacultyID	CourseID	Rating	Comments
1	1	1	1	5	Excellent teaching
2	1	1	7	4	Good pace
3	2	4	2	4	Very helpful
4	2	1	1	5	Clear explanations
5	3	2	3	5	Explains clearly
6	3	3	4	4	Practical examples
7	4	3	4	3	Needs more examples
8	4	6	5	4	Engaging labs
9	5	1	1	4	Good instructor
10	5	4	8	3	Fast paced
11	6	6	5	5	Best faculty
12	6	3	4	4	Well organised
13	7	1	1	4	Solid fundamentals
14	7	4	8	3	Could slow down
15	8	5	6	4	Friendly approach
16	9	1	7	5	Very inspiring
17	9	4	2	3	Average pace

18	10	2	3	4	Detailed notes
19	11	6	5	5	Great mentor
20	12	4	2	4	Helpful feedback

Table 12: Third Normal form data for feedback table

6. PRACTICAL 2 : DATABASE CREATION

6.1 Database Creation

This step creates the StudentInformationSystem database in Microsoft SQL Server. If a database with the same name already exists, it is removed to prevent conflicts. The newly created database is then selected for all subsequent operations.

SQL Script

```
IF DB_ID('StudentInformationSystem') IS NOT NULL
BEGIN
    ALTER DATABASE StudentInformationSystem SET SINGLE_USER WITH ROLLBACK
    IMMEDIATE;
    DROP DATABASE StudentInformationSystem;
END
GO
CREATE DATABASE StudentInformationSystem;
GO
USE StudentInformationSystem;
GO
```

```

1
2  -- DATABASE CREATION
3
4  IF DB_ID('StudentInformationSystem') IS NOT NULL
5  BEGIN
6      ALTER DATABASE StudentInformationSystem SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
7      DROP DATABASE StudentInformationSystem;
8  END
9  GO
10
11 CREATE DATABASE StudentInformationSystem;
12 GO
13
14 USE StudentInformationSystem;
15 GO
16
17
18 -- 1. TABLE CREATION
19
20 -- 1.1 Department

```

82 % No issues found

Messages

Commands completed successfully.

Completion time: 2026-07-20T19:15:28.5369128+05:45

Figure 2: Creating database

6.2 Table Creation

6.2.1 Department

The Department table stores information about academic departments. Each department is assigned a unique Department ID, along with its department name and the Head of Department (HOD).

SQL Script

CREATE TABLE Department

```

(
    DepartmentID INT IDENTITY(1,1) PRIMARY KEY,
    DepartmentName VARCHAR(100) NOT NULL UNIQUE,
    HOD VARCHAR(100) NOT NULL
);

```

```

18  -- 1. TABLE CREATION
19
20  -- 1.1 Department
21  CREATE TABLE Department
22  (
23      DepartmentID    INT IDENTITY(1,1) PRIMARY KEY,
24      DepartmentName  VARCHAR(100) NOT NULL UNIQUE,
25      HOD             VARCHAR(100) NOT NULL
26  );
27  GO

```

82 % No issues found

Messages

Commands completed successfully.

Completion time: 2026-07-20T19:17:13.4563937+05:45

Figure 3: Creation of the Department table.

6.2.2 Admin Table

The Admin table stores information about system administrators responsible for managing the Student Information Management System. It includes the administrator's name, email address, and assigned role. Each administrator is uniquely identified by an Admin ID.

SQL Script

```

CREATE TABLE Admin(
    AdminID    INT IDENTITY(1,1) PRIMARY KEY,
    AdminName  VARCHAR(100) NOT NULL,
    Email      VARCHAR(100) NOT NULL UNIQUE,
    Role       VARCHAR(50) DEFAULT 'System Administrator';
GO

```

```

29  -- 1.2 Admin
30  CREATE TABLE Admin
31  (
32      AdminID          INT IDENTITY(1,1) PRIMARY KEY,
33      AdminName        VARCHAR(100) NOT NULL,
34      Email            VARCHAR(100) NOT NULL UNIQUE,
35      Role             VARCHAR(50)  DEFAULT 'System Administrator'
36  );
37  GO
38
39  -- 1.3 Students

```

82 % No issues found

Messages

Commands completed successfully.

Completion time: 2026-07-20T19:47:31.3727161+05:45

Figure 4: Creation of the Admin table in Microsoft SQL Server.

6.2.3 Students Table

The Students table stores detailed information about students, including personal information, contact details, enrollment year, and their associated department and administrator. Foreign key constraints maintain relationships with the Department and Admin tables to ensure data integrity.

SQL Script

```

CREATE TABLE Students(
    StudentID    INT IDENTITY(1,1) PRIMARY KEY,
    FirstName    VARCHAR(50) NOT NULL,
    LastName     VARCHAR(50) NOT NULL,
    Gender       CHAR(1)    CHECK (Gender IN ('M','F','O')),
    DOB          DATE       NOT NULL,
    Email        VARCHAR(100) NOT NULL UNIQUE,
    Phone        VARCHAR(10) NOT NULL,
    Address      VARCHAR(200) NULL,
    EnrollmentYear INT      NOT NULL CHECK (EnrollmentYear BETWEEN
        2015 AND 2030),
    DepartmentID INT        NOT NULL,
    AdminID      INT        NOT NULL,

```



```

CONSTRAINT FK_Students_Department
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID),
CONSTRAINT FK_Students_Admin
    FOREIGN KEY (AdminID) REFERENCES Admin(AdminID));
GO

```

```

38
39 -- 1.3 Students
40
41 CREATE TABLE Students
42 (
43     StudentID      INT IDENTITY(1,1) PRIMARY KEY,
44     FirstName      VARCHAR(50) NOT NULL,
45     LastName       VARCHAR(50) NOT NULL,
46     Gender         CHAR(1) CHECK (Gender IN ('M', 'F', 'O')),
47     DOB            DATE NOT NULL,
48     Email          VARCHAR(100) NOT NULL UNIQUE,
49     Phone          VARCHAR(10) NOT NULL,
50     Address        VARCHAR(200) NULL,
51     EnrollmentYear INT NOT NULL CHECK (EnrollmentYear BETWEEN 2015 AND 2030),
52     DepartmentID   INT NOT NULL,
53     AdminID        INT NOT NULL,
54
55     CONSTRAINT FK_Students_Department
56         FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID),
57     CONSTRAINT FK_Students_Admin
58         FOREIGN KEY (AdminID) REFERENCES Admin(AdminID)
59 );
60 GO
61
82 %  No issues found  Ln: 56, Ch: 72
Messages
Commands completed successfully.
Completion time: 2026-07-20T19:50:53.9100531+05:45

```

Figure 5: Creation of the Students table.

6.2.4 Faculty Table

The Faculty table stores information about faculty members, including their contact details, department affiliation, administrator approval, and joining date. Foreign key relationships connect each faculty member to a department and an administrator.

SQL Script

```

CREATE TABLE Faculty
(
    FacultyID      INT IDENTITY(1,1) PRIMARY KEY,
    FacultyName    VARCHAR(100) NOT NULL,

```

```

Email      VARCHAR(100) NOT NULL UNIQUE,
Phone      VARCHAR(15) NOT NULL,
DepartmentID INT      NOT NULL,
AdminID    INT        NOT NULL,  -- admin who approved the appointment
JoiningDate DATE      DEFAULT GETDATE(),

CONSTRAINT FK_Faculty_Department
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID),
CONSTRAINT FK_Faculty_Admin
    FOREIGN KEY (AdminID) REFERENCES Admin(AdminID)
);
GO

```

```

62 -- 1.4 Faculty
63 CREATE TABLE Faculty
64 (
65     FacultyID INT IDENTITY(1,1) PRIMARY KEY,
66     FacultyName VARCHAR(100) NOT NULL,
67     Email VARCHAR(100) NOT NULL UNIQUE,
68     Phone VARCHAR(15) NOT NULL,
69     DepartmentID INT NOT NULL,
70     AdminID INT NOT NULL, -- admin who approved the appointment
71     JoiningDate DATE DEFAULT GETDATE(),
72
73     CONSTRAINT FK_Faculty_Department
74         FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID),
75     CONSTRAINT FK_Faculty_Admin
76         FOREIGN KEY (AdminID) REFERENCES Admin(AdminID)
77 );
78 GO
79

```

82 % No issues found Ln: 77, Ch: 3

Messages

Commands completed successfully.

Completion time: 2026-07-20T20:04:42.0598221+05:45

Figure 6: Creation of the Faculty table.

6.2.4 Courses Table

The Courses table stores details of all courses offered by the institution. It includes course name, credit hours, associated department, and the faculty member assigned to teach the course. Foreign key constraints establish relationships with the Department and Faculty tables.

SQL Script

```
CREATE TABLE Courses
```

```
(
```

```
    CourseID    INT IDENTITY(1,1) PRIMARY KEY,
    CourseName   VARCHAR(100) NOT NULL,
    Credits      INT      NOT NULL CHECK (Credits > 0),
    DepartmentID INT      NOT NULL,
    FacultyID    INT      NULL,
```

```
    CONSTRAINT FK_Courses_Department
```

```
        FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID),
```

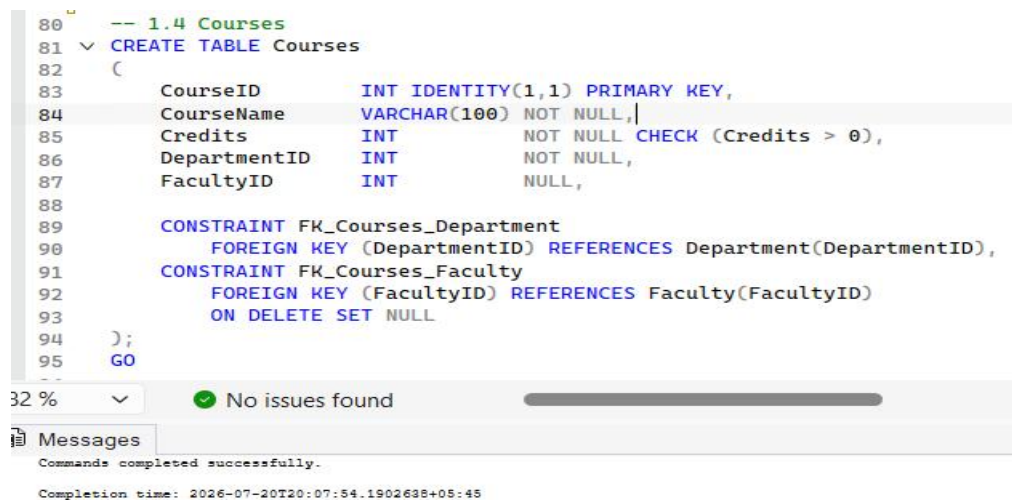
```
    CONSTRAINT FK_Courses_Faculty
```

```
        FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
```

```
        ON DELETE SET NULL
```

```
);
```

```
GO
```



```

80  -- 1.4 Courses
81  CREATE TABLE Courses
82  (
83      CourseID    INT IDENTITY(1,1) PRIMARY KEY,
84      CourseName   VARCHAR(100) NOT NULL,
85      Credits      INT      NOT NULL CHECK (Credits > 0),
86      DepartmentID INT      NOT NULL,
87      FacultyID    INT      NULL,
88
89      CONSTRAINT FK_Courses_Department
90          FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID),
91      CONSTRAINT FK_Courses_Faculty
92          FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
93          ON DELETE SET NULL
94  );
95  GO

```

32 % No issues found

Messages

Commands completed successfully.

Completion time: 2026-07-20T20:07:54.1902638+05:45

Figure 7: Creation of the Courses table.

6.2.5 Enrollment Table

The Enrollment table records the enrollment of students in courses. It stores information such as semester, grades, and enrollment date while maintaining relationships between students and courses through foreign keys. A unique constraint prevents duplicate enrollments for the same student, course, and semester

SQL Script

```
CREATE TABLE Enrollment
```

```
(
    EnrollmentID INT IDENTITY(1,1) PRIMARY KEY,
    StudentID INT NOT NULL,
    CourseID INT NOT NULL,
    Semester VARCHAR(20) NOT NULL,
    Grade CHAR(2) NULL CHECK (Grade IN ('A','A-','B+','B','B-','C+','C','D','F') OR Grade IS NULL),
    EnrollmentDate DATE DEFAULT GETDATE(),

    CONSTRAINT FK_Enrollment_Student
        FOREIGN KEY (StudentID) REFERENCES Students(StudentID) ON DELETE CASCADE,

    CONSTRAINT FK_Enrollment_Course
        FOREIGN KEY (CourseID) REFERENCES Courses(CourseID) ON DELETE CASCADE,

    CONSTRAINT UQ_Student_Course_Semester UNIQUE (StudentID, CourseID, Semester)
);
GO
```

```

96
97  -- 1.5 Enrollment
98  CREATE TABLE Enrollment
99  (
100      EnrollmentID INT IDENTITY(1,1) PRIMARY KEY,
101      StudentID INT NOT NULL,
102      CourseID INT NOT NULL,
103      Semester VARCHAR(20) NOT NULL,
104      Grade CHAR(2) NULL CHECK (Grade IN ('A','A-','B+','B','B-','C+','C','D','F') OR Grade IS NULL),
105      EnrollmentDate DATE DEFAULT GETDATE(),
106
107      CONSTRAINT FK_Enrollment_Student
108          FOREIGN KEY (StudentID) REFERENCES Students(StudentID) ON DELETE CASCADE,
109      CONSTRAINT FK_Enrollment_Course
110          FOREIGN KEY (CourseID) REFERENCES Courses(CourseID) ON DELETE CASCADE,
111      CONSTRAINT UQ_Student_Course_Semester UNIQUE (StudentID, CourseID, Semester)
112  );
113  GO
114
100 %  No issues found
Messages
Commands completed successfully.
Completion time: 2026-07-20T20:07:54.1902638+05:45

```

Figure 8: Creation of the Enrollment table.

6.2.6 Feedback Table

The Feedback table stores feedback submitted by students regarding faculty members and their courses. It records ratings, comments, and feedback dates while maintaining relationships with the Students, Faculty, and Courses tables using foreign keys.

SQL Script

```
CREATE TABLE Feedback
(
    FeedbackID    INT IDENTITY(1,1) PRIMARY KEY,
    StudentID     INT NOT NULL,
    FacultyID     INT NOT NULL,
    CourseID     INT NOT NULL,
    Rating        INT NOT NULL CHECK (Rating BETWEEN 1 AND 5),
    Comments      VARCHAR(200) NULL,
    FeedbackDate  DATE DEFAULT GETDATE(),

    CONSTRAINT FK_Feedback_Student FOREIGN KEY (StudentID)
REFERENCES Students(StudentID) ON DELETE CASCADE,
    CONSTRAINT FK_Feedback_Faculty FOREIGN KEY (FacultyID)
REFERENCES Faculty(FacultyID) ON DELETE CASCADE,
    CONSTRAINT FK_Feedback_Course FOREIGN KEY (CourseID)
REFERENCES Courses(CourseID)
);
GO
```

```

1144
115  -- 1.6 Feedback
116  CREATE TABLE Feedback
117  (
118      FeedbackID      INT IDENTITY(1,1) PRIMARY KEY,
119      StudentID       INT NOT NULL,
120      FacultyID       INT NOT NULL,
121      CourseID        INT NOT NULL,
122      Rating          INT NOT NULL CHECK (Rating BETWEEN 1 AND 5),
123      Comments        VARCHAR(200) NULL,
124      FeedbackDate    DATE DEFAULT GETDATE(),
125
126      CONSTRAINT FK_Feedback_Student FOREIGN KEY (StudentID) REFERENCES Students(StudentID) ON DELETE CASCADE,
127      CONSTRAINT FK_Feedback_Faculty FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID) ON DELETE CASCADE,
128      CONSTRAINT FK_Feedback_Course  FOREIGN KEY (CourseID)  REFERENCES Courses(CourseID)
129  );
130  GO
131
10 %  No issues found
Messages
Commands completed successfully.
Completion time: 2026-07-20T20:13:47.7356232+05:45

```

Figure 9: Creation of the Feedback table.

6.3 Sample Data Insertion

6.3.1 Department Data

Sample department records were inserted into the Department table to populate the database with initial department information for testing and demonstration purposes.

```

135
136  -- 2.1 Departments (4)
137  INSERT INTO Department (DepartmentName, HOD) VALUES
138  ('Computer Science', 'Dr. Anil Sharma'),
139  ('Mathematics',      'Dr. Rita Koirala'),
140  ('Cybersecurity',    'Prof. Bikash Thapa'),
141  ('Business Studies', 'Dr. Meena Adhikari');
142  GO
143
100 %  No issues found
Messages
(4 rows affected)
Completion time: 2026-07-21T19:17:20.3538044+05:45

```

Figure 10: Inserting sample records into the Department table.

6.3.2 Admin Data

Sample administrator records were inserted into the Admin table to represent users responsible for managing the system and monitoring academic operations.

```

143
144 -- 2.2 Admin (2) |
145 INSERT INTO Admin (AdminName, Email, Role) VALUES
146 ('Suman Shrestha', 'suman.admin@college.edu', 'System Administrator'),
147 ('Nisha Karki', 'nisha.admin@college.edu', 'Academic Registrar');
148 GO
149
100 %  No issues found  Ln: 144, Ch: 18
Messages
(2 rows affected)
Completion time: 2026-07-21T19:23:49.4019417+05:45

```

Figure 11: Inserting sample records into the Admin table.

6.3.3 Faculty Data

Sample faculty records were inserted into the Faculty table to demonstrate the storage of lecturer information and departmental assignments.

```

150 -- 2.3 Faculty (6)
151 INSERT INTO Faculty (FacultyName, Email, Phone, DepartmentID, AdminID, JoiningDate) VALUES
152 ('Dr. Anil Sharma', 'anil.sharma@college.edu', '9800000001', 1, 1, '2006-01-09'),
153 ('Dr. Rita Koirala', 'rita.koirala@college.edu', '9800000002', 2, 1, '2004-09-23'),
154 ('Prof. Bikash Thapa', 'bikash.thapa@college.edu', '9800000003', 3, 2, '2005-06-29'),
155 ('Ms. Sunita Lama', 'sunita.lama@college.edu', '9800000004', 1, 1, '2012-03-15'),
156 ('Mr. Deepak Basnet', 'deepak.basnet@college.edu', '9800000005', 4, 2, '2015-08-01'),
157 ('Dr. Kabita Nepal', 'kabita.nepal@college.edu', '9800000006', 3, 2, '2010-11-20');
158 GO
159
100 %  No issues found  Ln: 149, Ch: 1  SPC  CRLF  UTI
Messages
(6 rows affected)
Completion time: 2026-07-21T19:24:36.3577757+05:45

```

Figure 12: Inserting sample records into the Faculty table.

6.3.4 Courses Data

Sample course records were inserted into the Courses table to verify the storage of course information, credit values, and faculty assignments.


```

160  -- 2.4 Courses (8)
161  INSERT INTO Courses (CourseName, Credits, DepartmentID, FacultyID) VALUES
162  ('Database Management Systems', 4, 1, 1),
163  ('Data Structures', 4, 1, 4),
164  ('Calculus I', 3, 2, 2),
165  ('Network Security', 4, 3, 3),
166  ('Ethical Hacking', 3, 3, 6),
167  ('Business Communication', 2, 4, 5),
168  ('Web Technologies', 3, 1, 1),
169  ('Operating Systems', 4, 1, 4);
170  GO
171
100 %  No issues found  Ln: 158,
Messages
(8 rows affected)
Completion time: 2026-07-21T19:25:52.3902974+05:45

```

Figure 13: Inserting sample records into the Courses table.

6.3.5 Students Data

Sample student records were inserted into the Students table to verify the successful storage of student information and relationships with departments and administrators.

```

172  -- 2.5 Students (15)
173  INSERT INTO Students (FirstName, LastName, Gender, DOB, Email, Phone, Address, EnrollmentYear, DepartmentID, AdminID) VALUES
174  ('Ram', 'Bahadur', 'M', '2003-05-12', 'ram.bahadur@student.edu', '9811111111', 'Kathmandu', 2022, 1, 1),
175  ('Sita', 'Gurung', 'F', '2003-08-20', 'sita.gurung@student.edu', '9822222222', 'Lalitpur', 2022, 1, 1),
176  ('Hari', 'Shrestha', 'M', '2002-11-02', 'hari.shrestha@student.edu', '9833333333', 'Bhaktapur', 2021, 2, 2),
177  ('Gita', 'Rai', 'F', '2004-01-15', 'gita.raai@student.edu', '9844444444', 'Kathmandu', 2023, 3, 2),
178  ('Suresh', 'Karki', 'M', '2003-03-30', 'suresh.karki@student.edu', '9855555555', 'Pokhara', 2022, 1, 1),
179  ('Anita', 'Magar', 'F', '2004-06-18', 'anita.magar@student.edu', '9866666666', 'Kathmandu', 2023, 3, 2),
180  ('Bikash', 'Rai', 'M', '2004-02-20', 'bikash.raai@student.edu', '9877777777', 'Kathmandu', 2023, 1, 1),
181  ('Kabita', 'Thapa', 'F', '2003-09-09', 'kabita.thapa@student.edu', '9888888888', 'Chitwan', 2022, 4, 2),
182  ('Prakash', 'Adhikari', 'M', '2002-12-25', 'prakash.adhikari@student.edu', '9899999999', 'Kathmandu', 2021, 1, 1),
183  ('Manisha', 'Poudel', 'F', '2004-04-04', 'manisha.poudel@student.edu', '9801111111', 'Lalitpur', 2023, 2, 1),
184  ('Nabin', 'Tamang', 'M', '2003-07-07', 'nabin.tamang@student.edu', '9802222222', 'Bhaktapur', 2022, 3, 2),
185  ('Sabina', 'Lama', 'F', '2004-10-10', 'sabina.lama@student.edu', '9803333333', 'Kathmandu', 2023, 1, 1),
186  ('Rajesh', 'KC', 'M', '2002-05-05', 'rajesh.kc@student.edu', '9804444444', 'Pokhara', 2021, 4, 2),
187  ('Puja', 'Basnet', 'F', '2003-01-01', 'puja.basnet@student.edu', '9805555555', 'Kathmandu', 2022, 2, 1),
188  ('Deepak', 'Khadka', 'M', '2004-03-03', 'deepak.khadka@student.edu', '9806666666', 'Lalitpur', 2023, 3, 2);
189  GO
190
100 %  No issues found
Messages
(15 rows affected)
Completion time: 2026-07-21T19:26:40.5330456+05:45

```

Figure 14: Inserting sample records into the Students table.

6.3.6 Enrollment Data

Sample enrollment records were inserted into the Enrollment table to establish relationships between students and their enrolled courses across different semesters.

```

191  -- 2.6 Enrollment (20)
192  INSERT INTO Enrollment (StudentID, CourseID, Semester, Grade) VALUES
193  (1,1,'Fall 2024','A'), (1,7,'Spring 2025','B+'),
194  (2,2,'Fall 2024','B+'), (2,1,'Spring 2025','A-'),
195  (3,3,'Fall 2024','A-'), (3,4,'Spring 2025','B'),
196  (4,4,'Fall 2024','A'), (4,5,'Spring 2025','A-'),
197  (5,1,'Fall 2024','B'), (5,8,'Spring 2025','B+'),
198  (6,5,'Fall 2024','A'), (6,4,'Spring 2025','A'),
199  (7,1,'Fall 2024','A-'), (7,8,'Spring 2025','B'),
200  (8,6,'Fall 2024','B+'),
201  (9,7,'Fall 2024','A'), (9,2,'Spring 2025','B'),
202  (10,3,'Fall 2024','B'),
203  (11,5,'Fall 2024','A-'),
204  (12,2,'Fall 2024','B+');
205  GO
206
100 %  No issues found
Messages
(20 rows affected)
Completion time: 2026-07-21T19:27:41.9260971+05:45

```

Figure 15: Inserting sample records into the Enrollment table.

6.3.7 Feedback Data

Sample feedback records were inserted into the Feedback table to demonstrate the recording of student evaluations, ratings, and comments for faculty members and courses.

```

207  -- 2.7 Feedback (20)
208  INSERT INTO Feedback (StudentID, FacultyID, CourseID, Rating, Comments) VALUES
209  (1,1,1,5,'Excellent teaching'), (1,1,7,4,'Good pace'),
210  (2,4,2,4,'Very helpful'), (2,1,1,5,'Clear explanations'),
211  (3,2,3,5,'Explains clearly'), (3,3,4,4,'Practical examples'),
212  (4,3,4,3,'Needs more examples'), (4,6,5,4,'Engaging labs'),
213  (5,1,1,4,'Good instructor'), (5,4,8,3,'Fast paced'),
214  (6,6,5,5,'Best faculty'), (6,3,4,4,'Well organised'),
215  (7,1,1,4,'Solid fundamentals'), (7,4,8,3,'Could slow down'),
216  (8,5,6,4,'Friendly approach'),
217  (9,1,7,5,'Very inspiring'), (9,4,2,3,'Average pace'),
218  (10,2,3,4,'Detailed notes'),
219  (11,6,5,5,'Great mentor'),
220  (12,4,2,4,'Helpful feedback');
221  GO
222
100 %  No issues found
Messages
(20 rows affected)
Completion time: 2026-07-21T19:28:21.2176930+05:45

```

Figure 16: Inserting sample records into the Feedback table.

6.4 Data Verification

The inserted data was verified by executing SELECT statements on each table. This step confirms that all records were successfully inserted and that the database structure and relationships function correctly.

SQL Script

```
SELECT * FROM Department;
```

```
SELECT * FROM Faculty;
```

```
SELECT * FROM Courses;
```

```
SELECT * FROM Students;
```

```
SELECT * FROM Enrollment;
```

```
SELECT * FROM Feedback;
```

```
SELECT * FROM Admin;
```

```
GO
```

Results		Messages	
DepartmentID	DepartmentName	HOD	
1	Computer Science	Dr. Anil Sharma	
2	Mathematics	Dr. Rita Koirala	
3	Cybersecurity	Prof. Bikash Thapa	
4	Business Studies	Dr. Meena Adhikari	

FacultyID	FacultyName	Email	Phone	DepartmentID	AdminID	JoiningDate
1	Dr. Anil Sharma	anil.sharma@college.edu	9800000001	1	1	2006-01-09
2	Dr. Rita Koirala	rita.koirala@college.edu	9800000002	2	1	2004-09-23
3	Prof. Bikash Thapa	bikash.thapa@college.edu	9800000003	3	2	2005-06-29
4	Ms. Sunita Lama	sunita.lama@college.edu	9800000004	1	1	2012-03-15
5	Mr. Deepak Basnet	deepak.basnet@college.edu	9800000005	4	2	2015-08-01
6	Dr. Kabita Nepal	kabita.nepal@college.edu	9800000006	3	2	2010-11-20

CourseID	CourseName	Credits	DepartmentID	FacultyID
1	Database Management Systems	4	1	1
2	Data Structures	4	1	4
3	Calculus I	3	2	2
4	Network Security	4	3	3
5	Ethical Hacking	3	3	6
6	Business Communication	2	4	5
7	Web Technologies	3	1	1
8	Operating Systems	4	1	4

Figure 17: verification of values in department, faculty and course table

	StudentID	FirstName	LastName	Gender	DOB	Email	Phone	Address	EnrollmentYear	DepartmentID	AdminID
1	1	Ram	Bahadur	M	2003-05-12	ram.bahadur@student.edu	9811111111	Kathmandu	2022	1	1
2	2	Sita	Gurung	F	2003-08-20	sita.gurung@student.edu	9822222222	Lalitpur	2022	1	1
3	3	Hari	Shrestha	M	2002-11-02	hari.shrestha@student.edu	9833333333	Bhaktapur	2021	2	2
4	4	Gita	Rai	F	2004-01-15	gita.rai@student.edu	9844444444	Kathmandu	2023	3	2
5	5	Suresh	Karki	M	2003-03-30	suresh.karki@student.edu	9855555555	Pokhara	2022	1	1
6	6	Anita	Magar	F	2004-06-18	anita.magar@student.edu	9866666666	Kathmandu	2023	3	2
7	7	Bikash	Rai	M	2004-02-20	bikash.rai@student.edu	9877777777	Kathmandu	2023	1	1
8	8	Kabita	Thapa	F	2003-09-09	kabita.thapa@student.edu	9888888888	Chitwan	2022	4	2
9	9	Prakash	Adhikari	M	2002-12-25	prakash.adhikari@stude...	9899999999	Kathmandu	2021	1	1
10	10	Manisha	Poudel	F	2004-04-04	manisha.poudel@studen...	9801111111	Lalitpur	2023	2	1
11	11	Nabin	Tamang	M	2003-07-07	nabin.tamang@student.e...	9802222222	Bhaktapur	2022	3	2
12	12	Sabina	Lama	F	2004-10-10	sabina.lama@student.edu	9803333333	Kathmandu	2023	1	1
13	13	Rajesh	KC	M	2002-05-05	rajesh.kc@student.edu	9804444444	Pokhara	2021	4	2
14	14	Puja	Basnet	F	2003-01-01	puja.basnet@student.edu	9805555555	Kathmandu	2022	2	1
15	15	Deepak	Khadka	M	2004-03-03	deepak.khadka@student...	9806666666	Lalitpur	2023	3	2

Figure 18: verification of value insertion in student table

	EnrollmentID	StudentID	CourseID	Semester	Grade	EnrollmentDate
1	1	1	1	Fall 2024	A	2026-07-21
2	2	1	7	Spring 2025	B+	2026-07-21
3	3	2	2	Fall 2024	B+	2026-07-21
4	4	2	1	Spring 2025	A-	2026-07-21
5	5	3	3	Fall 2024	A-	2026-07-21
6	6	3	4	Spring 2025	B	2026-07-21
7	7	4	4	Fall 2024	A	2026-07-21
8	8	4	5	Spring 2025	A-	2026-07-21
9	9	5	1	Fall 2024	B	2026-07-21
10	10	5	8	Spring 2025	B+	2026-07-21
11	11	6	5	Fall 2024	A	2026-07-21
12	12	6	4	Spring 2025	A	2026-07-21
13	13	7	1	Fall 2024	A-	2026-07-21
14	14	7	8	Spring 2025	B	2026-07-21
15	15	8	6	Fall 2024	B+	2026-07-21
16	16	9	7	Fall 2024	A	2026-07-21
17	17	9	2	Spring 2025	B	2026-07-21
18	18	10	3	Fall 2024	B	2026-07-21
19	19	11	5	Fall 2024	A-	2026-07-21
20	20	12	2	Fall 2024	B+	2026-07-21

Figure 19: value insertion verification in enrollment table

	FeedbackID	StudentID	FacultyID	CourseID	Rating	Comments	FeedbackDate
1	1	1	1	1	5	Excellent teaching	2026-07-21
2	2	1	1	7	4	Good pace	2026-07-21
3	3	2	4	2	4	Very helpful	2026-07-21
4	4	2	1	1	5	Clear explanations	2026-07-21
5	5	3	2	3	5	Explains clearly	2026-07-21
6	6	3	3	4	4	Practical examples	2026-07-21
7	7	4	3	4	3	Needs more examples	2026-07-21
8	8	4	6	5	4	Engaging labs	2026-07-21
9	9	5	1	1	4	Good instructor	2026-07-21
10	10	5	4	8	3	Fast paced	2026-07-21
11	11	6	6	5	5	Best faculty	2026-07-21
12	12	6	3	4	4	Well organised	2026-07-21
13	13	7	1	1	4	Solid fundamentals	2026-07-21
14	14	7	4	8	3	Could slow down	2026-07-21
15	15	8	5	6	4	Friendly approach	2026-07-21
16	16	9	1	7	5	Very inspiring	2026-07-21
17	17	9	4	2	3	Average pace	2026-07-21
18	18	10	2	3	4	Detailed notes	2026-07-21
19	19	11	6	5	5	Great mentor	2026-07-21
20	20	12	4	2	4	Helpful feedback	2026-07-21

Figure 20: values inserion verification in feedback table

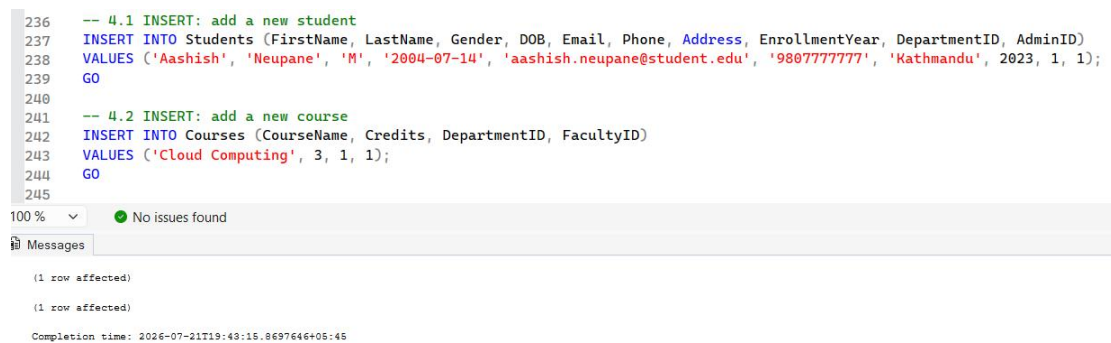
	AdminID	AdminName	Email	Role
1	1	Suman Shrestha	suman.admin@college.edu	System Administrator
2	2	Nisha Karki	nisha.admin@college.edu	Academic Registrar

Figure 21: verification of inserted recrd in admin table

7. PRACTICAL 3: CRUD OPERATIONS & QUERIES

7.1 INSERT

The INSERT operation is used to add new records to the database. In this project, new student and course records are inserted into the Students and Courses tables to demonstrate how data can be added while maintaining the defined relationships and constraints.



```

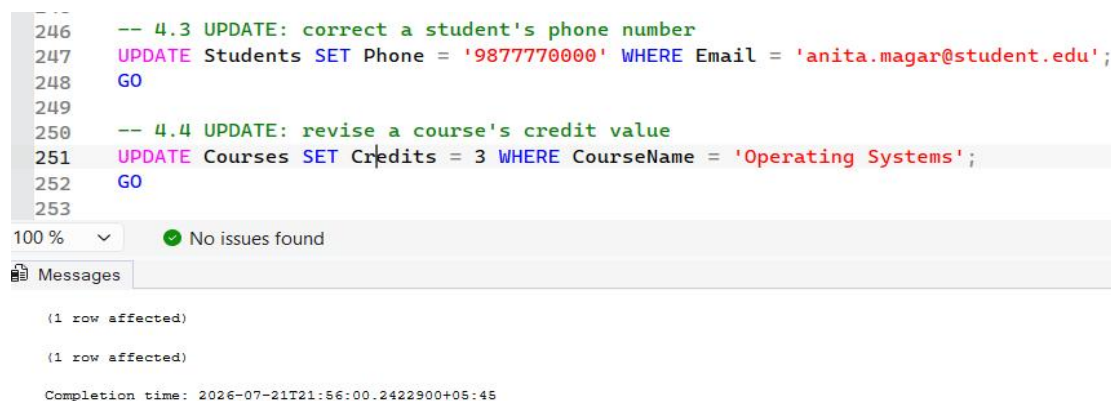
236 -- 4.1 INSERT: add a new student
237 INSERT INTO Students (FirstName, LastName, Gender, DOB, Email, Phone, Address, EnrollmentYear, DepartmentID, AdminID)
238 VALUES ('Aashish', 'Neupane', 'M', '2004-07-14', 'aashish.neupane@student.edu', '9807777777', 'Kathmandu', 2023, 1, 1);
239 GO
240
241 -- 4.2 INSERT: add a new course
242 INSERT INTO Courses (CourseName, Credits, DepartmentID, FacultyID)
243 VALUES ('Cloud Computing', 3, 1, 1);
244 GO
245
100 %  No issues found
Messages
(1 row affected)
(1 row affected)
Completion time: 2026-07-21T19:43:15.8697646+05:45

```

Figure 22 Inserting new records into the Students and Courses tables.

7.2 UPDATE

The UPDATE operation is used to modify existing records in the database. In this project, the student's phone number and the credit value of a course are updated to demonstrate how existing data can be edited while preserving data integrity.



```

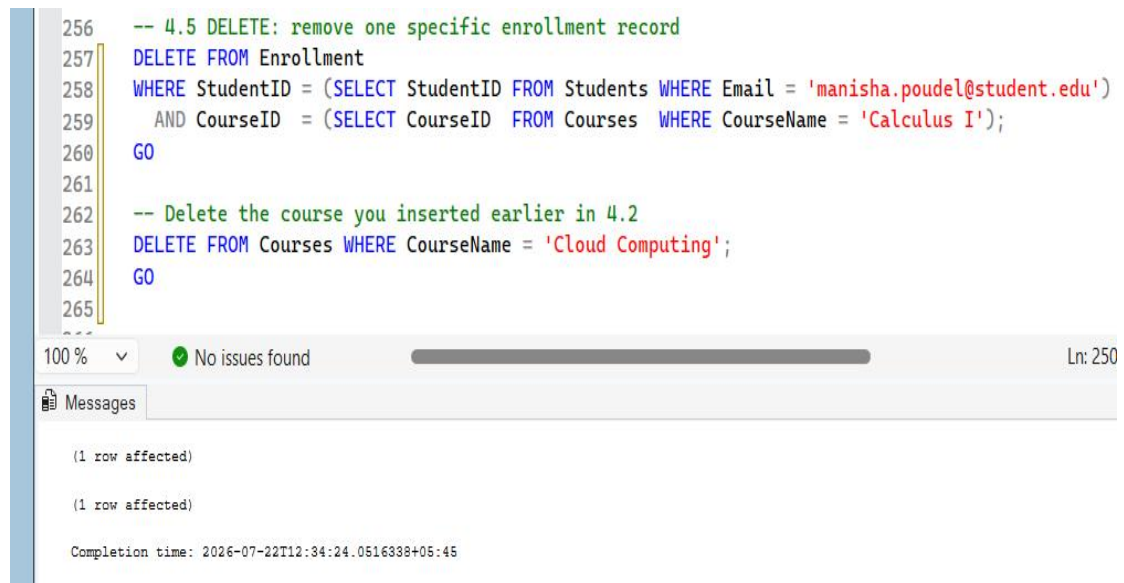
246 -- 4.3 UPDATE: correct a student's phone number
247 UPDATE Students SET Phone = '9877770000' WHERE Email = 'anita.magar@student.edu';
248 GO
249
250 -- 4.4 UPDATE: revise a course's credit value
251 UPDATE Courses SET Credits = 3 WHERE CourseName = 'Operating Systems';
252 GO
253
100 %  No issues found
Messages
(1 row affected)
(1 row affected)
Completion time: 2026-07-21T21:56:00.2422900+05:45

```


Figure 23: Updating existing records in the Students and Courses tables.

7.3 DELETE

This query removes a specific enrollment record by identifying the student and course through sub-queries. It demonstrates the DELETE operation used to remove unwanted or outdated records from the database.



```

256  -- 4.5 DELETE: remove one specific enrollment record
257  DELETE FROM Enrollment
258  WHERE StudentID = (SELECT StudentID FROM Students WHERE Email = 'manisha.poudel@student.edu')
259        AND CourseID = (SELECT CourseID FROM Courses WHERE CourseName = 'Calculus I');
260  GO
261
262  -- Delete the course you inserted earlier in 4.2
263  DELETE FROM Courses WHERE CourseName = 'Cloud Computing';
264  GO
265
  
```

100 % No issues found Ln: 250

Messages

(1 row affected)

(1 row affected)

Completion time: 2026-07-22T12:34:24.0516338+05:45

Figure 24: Deleting a selected enrollment record from the Enrollment table and recently inserted courses

7.4 SELECT with JOIN

This query retrieves student information together with the courses they have enrolled in, including the semester and grade. It uses INNER JOIN operations to combine data from the Students, Enrollment, and Courses tables.

SQL Script

```

SELECT S.StudentID, S.FirstName + ' ' + S.LastName AS StudentName,
       C.CourseName, E.Semester, E.Grade
FROM Students S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Courses C   ON E.CourseID = C.CourseID
ORDER BY S.StudentID;
  
```

GO

100 % No issues found

Results Messages

	StudentID	StudentName	CourseName	Semester	Grade
1	1	Ram Bahadur	Database Management Systems	Fall 2024	A
2	1	Ram Bahadur	Web Technologies	Spring 2025	B+
3	2	Sita Gurung	Database Management Systems	Spring 2025	A-
4	2	Sita Gurung	Data Structures	Fall 2024	B+
5	3	Hari Shrestha	Calculus I	Fall 2024	A-
6	3	Hari Shrestha	Network Security	Spring 2025	B
7	4	Gita Rai	Network Security	Fall 2024	A
8	4	Gita Rai	Ethical Hacking	Spring 2025	A-
9	5	Suresh Karki	Database Management Systems	Fall 2024	B
10	5	Suresh Karki	Operating Systems	Spring 2025	B+
11	6	Anita Magar	Network Security	Spring 2025	A
12	6	Anita Magar	Ethical Hacking	Fall 2024	A
13	7	Bikash Rai	Database Management Systems	Fall 2024	A-
14	7	Bikash Rai	Operating Systems	Spring 2025	B
15	8	Kabita Thapa	Business Communication	Fall 2024	B+
16	9	Prakash Adh...	Data Structures	Spring 2025	B
17	9	Prakash Adh...	Web Technologies	Fall 2024	A
18	10	Manisha Po...	Calculus I	Fall 2024	B
19	11	Nabin Tama...	Ethical Hacking	Fall 2024	A-
20	12	Sabina Lama	Data Structures	Fall 2024	B+

Figure 25: Retrieving student details with enrolled courses using INNER JOIN.

7.5 Search Conditions

The following SQL queries demonstrate the use of LIKE, BETWEEN, and IN operators to search and retrieve records based on specific conditions.

- The LIKE operator performs pattern matching,
- BETWEEN filters records within a specified range,
- IN retrieves records that match one or more specified values.

These queries help users efficiently search and filter data stored in the database.

SQLQuery2.s...kharel (71))

```

268
269 -- 4.7 LIKE: students whose first name starts with 'S'
270 SELECT StudentID, FirstName, LastName FROM Students WHERE FirstName LIKE 'S%';
271 GO
272 -- 4.8 BETWEEN: students born between 2003 and 2004
273 SELECT StudentID, FirstName, LastName, DOB FROM Students WHERE DOB BETWEEN '2003-01-01' AND '2004-12-31';
274 GO
275 -- 4.9 IN: courses in a given set of departments
276 SELECT CourseID, CourseName FROM Courses WHERE DepartmentID IN (1,3);
277 GO

```

100 % No issues found

Results Messages

	StudentID	FirstName	LastName
1	2	Sita	Gurung
2	5	Suresh	Karki
3	12	Sabina	Lama

	StudentID	FirstName	LastName	DOB
1	1	Ram	Bahadur	2003-05-12
2	2	Sita	Gurung	2003-08-20
3	4	Gita	Rai	2004-01-15
4	5	Suresh	Karki	2003-03-30
5	6	Anita	Magar	2004-06-18
6	7	Bikash	Rai	2004-02-20
7	8	Kabita	Thapa	2003-09-09
8	10	Manisha	Poudel	2004-04-04
9	11	Nabin	Tamang	2003-07-07
10	12	Sabina	Lama	2004-10-10
11	14	Puja	Basnet	2003-01-01
12	15	Deepak	Khadka	2004-03-03
13	16	Aashish	Neupane	2004-07-14

	CourseID	CourseName
1	1	Database Management Systems
2	2	Data Structures
3	4	Network Security
4	5	Ethical Hacking
5	7	Web Technologies
6	8	Operating Systems
7	9	Cloud Computing

Query executed successfully. (local)\SQ

Figure 26: Executing search queries using the LIKE, BETWEEN, and IN operators.

7.6 ORDER BY, DISTINCT, TOP, and Aliases

The following SQL queries demonstrate different techniques for retrieving and presenting data.

- ORDER BY sorts records,
- DISTINCT removes duplicate values,
- TOP limits the number of returned records,
- Aliases improve the readability of query results.

```

279  -- 4.10 ORDER BY: students ordered by enrollment year (descending)
280  SELECT FirstName, LastName, EnrollmentYear FROM Students ORDER BY EnrollmentYear DESC;
281  GO
282

```

100 % No issues found

Results Messages

	FirstName	LastName	EnrollmentYear
1	Gita	Rai	2023
2	Anita	Magar	2023
3	Bikash	Rai	2023
4	Manisha	Poudel	2023
5	Sabina	Lama	2023
6	Deepak	Khadka	2023
7	Aashish	Neupane	2023
8	Puja	Basnet	2022
9	Nabin	Tamang	2022
10	Kabita	Thapa	2022
11	Suresh	Karki	2022
12	Ram	Bahadur	2022
13	Sita	Gurung	2022
14	Hari	Shrestha	2021
15	Prakash	Adhikari	2021
16	Rajesh	KC	2021

Figure 27: Executing SQL queries using ORDER BY

```

283  -- 4.11 DISTINCT: distinct departments that offer courses
284  SELECT DISTINCT D.DepartmentName
285  FROM Courses C JOIN Department D ON C.DepartmentID = D.DepartmentID;
286  GO
287

```

100 % No issues found

Results Messages

	DepartmentName
1	Computer Science
2	Mathematics
3	Cybersecurity
4	Business Studies

Figure 28: Executing SQL queries using DISTINCT


```

207
290  -- 4.12 TOP: top 5 highest-credit courses
291  SELECT TOP 5 CourseName, Credits
292  FROM Courses
293  ORDER BY Credits DESC, CourseName ASC;
294  GO
295

```

100 % No issues found

Results Messages

	CourseName	Credits
1	Data Structures	4
2	Database Management Systems	4
3	Network Security	4
4	Calculus I	3
5	Cloud Computing	3

Figure 29: Executing SQL queries using TOP

```

291
292  -- 4.13 Aliases: shortened, readable output
293  SELECT S.FirstName AS FName, S.LastName AS LName, C.CourseName AS Course
294  FROM Students AS S
295  INNER JOIN Enrollment AS E ON S.StudentID = E.StudentID
296  INNER JOIN Courses AS C ON E.CourseID = C.CourseID;
297  GO
298

```

100 % No issues found

Results Messages

	FName	LName	Course
1	Ram	Bahadur	Database Management Systems
2	Ram	Bahadur	Web Technologies
3	Sita	Gurung	Database Management Systems
4	Sita	Gurung	Data Structures
5	Hari	Shrestha	Calculus I
6	Hari	Shrestha	Network Security
7	Gita	Rai	Network Security
8	Gita	Rai	Ethical Hacking
9	Suresh	Karki	Database Management Systems
10	Suresh	Karki	Operating Systems
11	Anita	Magar	Network Security
12	Anita	Magar	Ethical Hacking
13	Bikash	Rai	Database Management Systems
14	Bikash	Rai	Operating Systems
15	Kabita	Thapa	Business Communication
16	Prakash	Adhikari	Data Structures
17	Prakash	Adhikari	Web Technologies
18	Manisha	Poudel	Calculus I
19	Nabin	Tamang	Ethical Hacking
20	Sabina	Lama	Data Structures

Figure 30: Executing SQL queries using Aliases.

8. PRACTICAL 4: RELATIONAL ALGEBRA & TRANSACTIONS

8.1 Relational Algebra Operators

The following relational algebra operations were implemented using SQL queries to demonstrate different methods of retrieving and manipulating data within the database.

- Selection (σ): Selects records that satisfy a specified condition, such as retrieving only female students.
- Projection (π): Retrieves only the required attributes (columns) from a table, reducing unnecessary data.
- Join ($\bowtie$): Combines related data from multiple tables based on matching keys to produce meaningful information.
- Union ($\cup$): Merges the results of two queries into a single result set while eliminating duplicate records.
- Intersection ($\cap$): Returns only the records that are common to both query results.
- Difference ($-$): Retrieves records that exist in the first query but not in the second query.
- Cartesian Product ($\times$): Produces all possible combinations of rows between two tables using a cross join.

```

303 -- 5.1 SELECTION (sigma): female students only
304 SELECT * FROM Students WHERE Gender = 'F';
305 GO
306
307 -- 5.2 PROJECTION (pi): only names
308 SELECT FirstName, LastName FROM Students;
309 GO
310

```

100 % ● No issues found

	StudentID	FirstName	LastName	Gender	DOB	Email	Phone	Address	EnrollmentYear	DepartmentID	AdminID
1	2	Sita	Gurung	F	2003-08-20	sita.gurung@student.edu	9822222222	Lalitpur	2022	1	1
2	4	Gita	Rai	F	2004-01-15	gita.rai@student.edu	9844444444	Kathmandu	2023	3	2
3	6	Anita	Magar	F	2004-06-18	anita.magar@student.edu	9877770000	Kathmandu	2023	3	2
4	8	Kabita	Thapa	F	2003-09-09	kabita.thapa@student.edu	9888888888	Chitwan	2022	4	2
5	10	Manisha	Poudel	F	2004-04-04	manisha.poudel@student.edu	9801111111	Lalitpur	2023	2	1
6	12	Sabina	Lama	F	2004-10-10	sabina.lama@student.edu	9803333333	Kathmandu	2023	1	1
7	14	Puja	Basnet	F	2003-01-01	puja.basnet@student.edu	9805555555	Kathmandu	2022	2	1

	FirstName	LastName
1	Ram	Bahadur
2	Sita	Gurung
3	Hari	Shrestha
4	Gita	Rai
5	Suresh	Karki
6	Anita	Magar
7	Bikash	Rai
8	Kabita	Thapa
9	Prakash	Adhikari
10	Manisha	Poudel
11	Nabin	Tamang
12	Sabina	Lama
13	Rajesh	KC
14	Puja	Basnet
15	Deepak	Khadka
16	Aashish	Neupane

Figure 31: Execution of relational algebra operations that include selection and projection

```

310
311 -- 5.3 JOIN: students with the courses they take
312 SELECT S.FirstName, C.CourseName
313 FROM Students S JOIN Enrollment E ON S.StudentID = E.StudentID
314 JOIN Courses C ON E.CourseID = C.CourseID;
315 GO
316

```

100 % No issues found

	FirstName	CourseName
1	Ram	Database Management Systems
2	Ram	Web Technologies
3	Sita	Database Management Systems
4	Sita	Data Structures
5	Hari	Calculus I
6	Hari	Network Security
7	Gita	Network Security
8	Gita	Ethical Hacking
9	Suresh	Database Management Systems
10	Suresh	Operating Systems
11	Anita	Network Security
12	Anita	Ethical Hacking
13	Bikash	Database Management Systems
14	Bikash	Operating Systems
15	Kabita	Business Communication
16	Prakash	Data Structures
17	Prakash	Web Technologies
18	Manisha	Calculus I
19	Nabin	Ethical Hacking
20	Sabina	Data Structures

Figure 32: Execution of relational algebra operations that include join

```

310
317 -- 5.4 UNION: combined list of student and faculty first names
318 SELECT FirstName AS PersonName FROM Students
319 UNION
320 SELECT FacultyName FROM Faculty;
321 GO
322

```

100 % No issues found

	PersonName
1	Aashish
2	Anita
3	Bikash
4	Deepak
5	Dr. Anil Sharma
6	Dr. Kabita Nepal
7	Dr. Rita Koirala
8	Gita
9	Hari
10	Kabita
11	Manisha
12	Mr. Deepak Basnet
13	Ms. Sunita Lama
14	Nabin
15	Prakash
16	Prof. Bikash Thapa
17	Puja
18	Rajesh
19	Ram
20	Sabina
21	Sita
22	Suresh

Figure 33: Execution of relational algebra operations that include union

```

328
329 -- 5.6 DIFFERENCE: students who have never submitted feedback
330 SELECT StudentID FROM Students
331 EXCEPT
332 SELECT StudentID FROM Feedback;
333 GO
334
335 -- 5.7 CARTESIAN PRODUCT: every department paired with every admin (illustrative only)
336 SELECT D.DepartmentName, A.AdminName FROM Department D CROSS JOIN Admin A;
337 GO
338
339

```

100 % No issues found

Results Messages

	StudentID
1	13
2	14
3	15
4	16

	DepartmentName	AdminName
1	Computer Science	Suman Shrestha
2	Mathematics	Suman Shrestha
3	Cybersecurity	Suman Shrestha
4	Business Studies	Suman Shrestha
5	Computer Science	Nisha Karki
6	Mathematics	Nisha Karki
7	Cybersecurity	Nisha Karki
8	Business Studies	Nisha Karki

Figure 34: Execution of relational algebra operations that include difference and Cartesian product

8.2 Transaction Management

The following transaction management operations were performed to demonstrate how SQL maintains data consistency and integrity during database transactions.

- **COMMIT:** Permanently saves all changes made during a transaction to the database.
- **ROLLBACK:** Cancels all changes made during a transaction and restores the database to its previous state.
- **SAVEPOINT:** Creates a checkpoint within a transaction, allowing selected changes to be rolled back without affecting the entire transaction.
- **Proof:** Verify that the transaction operations were executed correctly by confirming the results of COMMIT, ROLLBACK, and SAVEPOINT.

```

341 |
342 | -- COMMIT: change is saved permanently
343 | BEGIN TRANSACTION;
344 |     INSERT INTO Students (FirstName, LastName, Gender, DOB, Email, Phone, Address, EnrollmentYear, DepartmentID, AdminID)
345 |     VALUES ('Sarita', 'Bhandari', 'F', '2004-08-08', 'sarital.bhandari@student.edu', '9808888888', 'Kathmandu', 2023, 1, 1);
346 | COMMIT;
347 | GO
348 |
349 | -- ROLLBACK: change is undone
350 | BEGIN TRANSACTION;
351 |     INSERT INTO Students (FirstName, LastName, Gender, DOB, Email, Phone, Address, EnrollmentYear, DepartmentID, AdminID)
352 |     VALUES ('Test', 'User', 'M', '2004-01-01', 'test.user@student.edu', '9800000000', 'Kathmandu', 2023, 1, 1);
353 | ROLLBACK;
354 | GO
355 |
356 | -- SAVEPOINT: partial rollback within a transaction
357 | BEGIN TRANSACTION;
358 |     UPDATE Students SET Phone = '9809999999' WHERE Email = 'sarita.bhandari@student.edu';
359 |     SAVE TRANSACTION BeforeGradeUpdate;
360 |     UPDATE Enrollment SET Grade = 'F' WHERE StudentID = 1 AND CourseID = 1; -- accidental change
361 |     ROLLBACK TRANSACTION BeforeGradeUpdate; -- undo only the grade change
362 | COMMIT;
363 | GO

```

100 % No issues found

Messages

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

Completion time: 2026-07-22T06:44:40.1982778+05:45

Figure 35: Execution of transaction management operations using COMMIT, ROLLBACK, and SAVEPOINT.

```

365 | -- Proof: 'Sarita' exists, 'Test' does not, grade for StudentID 1/CourseID 1 unchanged
366 | SELECT * FROM Students WHERE FirstName IN ('Sarita', 'Test');
367 | SELECT * FROM Enrollment WHERE StudentID = 1 AND CourseID = 1;
368 | GO
369 |

```

100 % No issues found

Results Messages

	StudentID	FirstName	LastName	Gender	DOB	Email	Phone	Address	EnrollmentYear	DepartmentID	AdminID
1	17	Sarita	Bhandari	F	2004-08-08	sarita.bhandari@student.edu	9809999999	Kathmandu	2023	1	1
2	20	Sarita	Bhandari	F	2004-08-08	sarita1.bhandari@student.edu	9808888888	Kathmandu	2023	1	1

	EnrollmentID	StudentID	CourseID	Semester	Grade	EnrollmentDate
1	1	1	1	Fall 2024	A	2026-07-21

Figure 36: Verification of transaction management operations.

9. PRACTICAL 5: DATABASE OPERATIONS

9.1 SQL Joins

The following SQL join operations combine data from multiple tables to retrieve related information. Different types of joins are used to display matching records, unmatched records, and all possible combinations of data.

- **INNER JOIN:** Retrieves only the matching records from related tables.
- **LEFT JOIN:** Returns all records from the left table and matching records from the right table.
- **RIGHT JOIN:** Returns all records from the right table and matching records from the left table.
- **FULL JOIN:** Returns all matching and non-matching records from both tables.
- **SELF JOIN:** Joins a table with itself to compare related records.
- **CROSS JOIN:** Produces every possible combination of rows from two tables.

```

373
374  -- 6.1 INNER JOIN: students with their enrolled courses and faculty
375  SELECT S.FirstName, S.LastName, C.CourseName, F.FacultyName, E.Grade
376  FROM Students S
377  INNER JOIN Enrollment E ON S.StudentID = E.StudentID
378  INNER JOIN Courses C    ON E.CourseID = C.CourseID
379  INNER JOIN Faculty F    ON C.FacultyID = F.FacultyID;
380  GO
381

```

100 % ✔ No issues found

	FirstName	LastName	CourseName	FacultyName	Grade
1	Ram	Bahadur	Database Management Systems	Dr. Anil Sharma	A
2	Ram	Bahadur	Web Technologies	Dr. Anil Sharma	B+
3	Sita	Gurung	Data Structures	Ms. Sunita Lama	B+
4	Sita	Gurung	Database Management Systems	Dr. Anil Sharma	A-
5	Hari	Shrestha	Calculus I	Dr. Rita Koirala	A-
6	Hari	Shrestha	Network Security	Prof. Bikash Thapa	B
7	Gita	Rai	Network Security	Prof. Bikash Thapa	A
8	Gita	Rai	Ethical Hacking	Dr. Kabita Nepal	A-
9	Suresh	Karki	Database Management Systems	Dr. Anil Sharma	B
10	Suresh	Karki	Operating Systems	Ms. Sunita Lama	B+
11	Anita	Magar	Ethical Hacking	Dr. Kabita Nepal	A
12	Anita	Magar	Network Security	Prof. Bikash Thapa	A
13	Bikash	Rai	Database Management Systems	Dr. Anil Sharma	A-
14	Bikash	Rai	Operating Systems	Ms. Sunita Lama	B
15	Kabita	Thapa	Business Communication	Mr. Deepak Basnet	B+
16	Prakash	Adhikari	Web Technologies	Dr. Anil Sharma	A
17	Prakash	Adhikari	Data Structures	Ms. Sunita Lama	B
18	Manisha	Poudel	Calculus I	Dr. Rita Koirala	B
19	Nabin	Tamang	Ethical Hacking	Dr. Kabita Nepal	A-
20	Sabina	Lama	Data Structures	Ms. Sunita Lama	B+

Figure 37: Executing SQL join operations using inner join

```

381
382 -- 6.2 LEFT JOIN: students who are NOT enrolled in any course
383 SELECT S.StudentID, S.FirstName, S.LastName
384 FROM Students S
385 LEFT JOIN Enrollment E ON S.StudentID = E.StudentID
386 WHERE E.EnrollmentID IS NULL;
387 GO
388

```

100 % No issues found Ln:

	StudentID	FirstName	LastName
1	13	Rajesh	KC
2	14	Puja	Basnet
3	15	Deepak	Khadka
4	16	Aashish	Neupane
5	17	Sarita	Bhandari
6	20	Sarita	Bhandari

Figure 38: Executing SQL join operation using left join

```

389 -- 6.3 RIGHT JOIN: courses that have NOT been taken by any student
390 SELECT C.CourseID, C.CourseName
391 FROM Enrollment E
392 RIGHT JOIN Courses C ON E.CourseID = C.CourseID
393 WHERE E.EnrollmentID IS NULL;
394 GO
395

```

100 % No issues found Ln: 400, Ch: 4

	CourseID	CourseName
1	9	Cloud Computing

Figure 39: Executing SQL join operation using right join

```

403 -- 6.5 SELF JOIN: compare grades of different students in the same course
404 SELECT A.StudentID AS Student1, B.StudentID AS Student2, A.CourseID, A.Grade AS Grade1, B.Grade AS Grade2
405 FROM Enrollment A
406 INNER JOIN Enrollment B ON A.CourseID = B.CourseID AND A.StudentID < B.StudentID;
407 GO
408

```

100 % No issues found

	Student1	Student2	CourseID	Grade1	Grade2
1	1	2	1	A	A-
2	1	5	1	A	B
3	1	7	1	A	A-
4	1	9	7	B+	A
5	2	9	2	B+	B
6	2	12	2	B+	B+
7	2	5	1	A-	B
8	2	7	1	A-	A-
9	3	10	3	A-	B
10	3	4	4	B	A
11	3	6	4	B	A
12	4	6	4	A	A
13	4	6	5	A-	A
14	4	11	5	A-	A-
15	5	7	1	B	A-
16	5	7	8	B+	B
17	6	11	5	A	A-
18	9	12	2	B	B+

Figure 40: Executing SQL join operation using self join

```

396 -- 6.4 FULL JOIN: complete picture of students and courses whether matched or not
397 SELECT S.FirstName, C.CourseName
398 FROM Students S
399 FULL JOIN Enrollment E ON S.StudentID = E.StudentID
400 FULL JOIN Courses C ON E.CourseID = C.CourseID;
401 GO

```

100 % No issues found Ln: 395, Ch: 1 SPC CRLF

Results Messages

	FirstName	CourseName
1	Ram	Database Management Systems
2	Ram	Web Technologies
3	Sita	Database Management Systems
4	Sita	Data Structures
5	Hari	Calculus I
6	Hari	Network Security
7	Gita	Network Security
8	Gita	Ethical Hacking
9	Suresh	Database Management Systems
10	Suresh	Operating Systems
11	Anita	Network Security
12	Anita	Ethical Hacking
13	Bikash	Database Management Systems
14	Bikash	Operating Systems
15	Kabita	Business Communication
16	Prakash	Data Structures
17	Prakash	Web Technologies
18	Manisha	Calculus I
19	Nabin	Ethical Hacking
20	Sabina	Data Structures
21	Rajesh	NULL
22	Puja	NULL
23	Deepak	NULL
24	Aashish	NULL
25	Sarita	NULL
26	Sarita	NULL
27	NULL	Cloud Computing

Figure 41: Executing SQL join operation using full join

```

409 -- 6.6 CROSS JOIN: every course paired with every faculty (for planning purposes)
410 SELECT C.CourseName, F.FacultyName FROM Courses C CROSS JOIN Faculty F;
411 GO
412

```

100 % No issues found

Results Messages

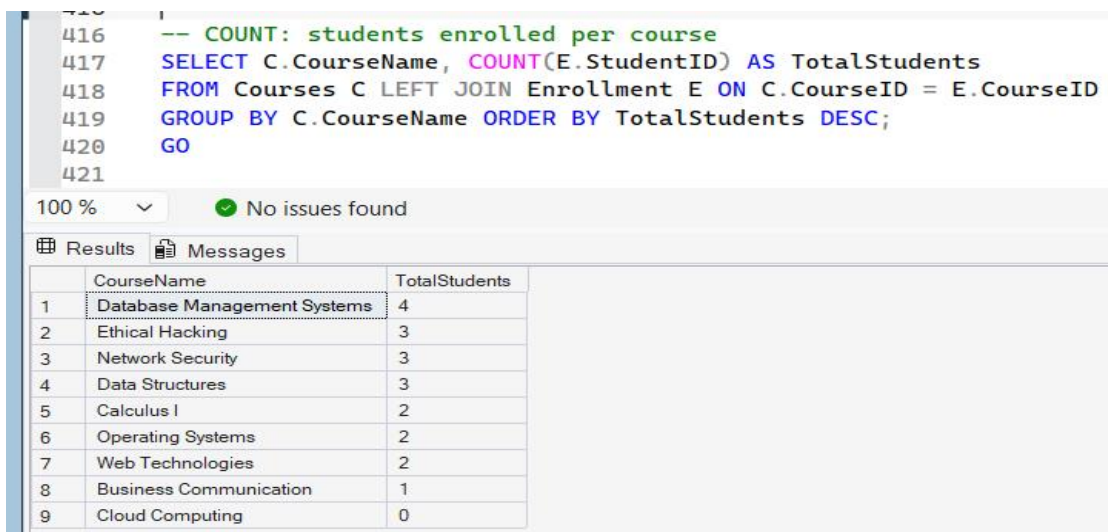
	CourseName	FacultyName
1	Database Management Systems	Dr. Anil Sharma
2	Data Structures	Dr. Anil Sharma
3	Calculus I	Dr. Anil Sharma
4	Network Security	Dr. Anil Sharma
5	Ethical Hacking	Dr. Anil Sharma
6	Business Communication	Dr. Anil Sharma
7	Web Technologies	Dr. Anil Sharma
8	Operating Systems	Dr. Anil Sharma
9	Cloud Computing	Dr. Anil Sharma
10	Database Management Systems	Dr. Rita Koirala
11	Data Structures	Dr. Rita Koirala
12	Calculus I	Dr. Rita Koirala
13	Network Security	Dr. Rita Koirala
14	Ethical Hacking	Dr. Rita Koirala
15	Business Communication	Dr. Rita Koirala
16	Web Technologies	Dr. Rita Koirala
17	Operating Systems	Dr. Rita Koirala
18	Cloud Computing	Dr. Rita Koirala
19	Database Management Systems	Prof. Bikash Thapa
20	Data Structures	Prof. Bikash Thapa
21	Calculus I	Prof. Bikash Thapa
22	Network Security	Prof. Bikash Thapa
23	Ethical Hacking	Prof. Bikash Thapa
24	Business Communication	Prof. Bikash Thapa
25	Web Technologies	Prof. Bikash Thapa
26	Operating Systems	Prof. Bikash Thapa
27	Cloud Computing	Prof. Bikash Thapa
28	Database Management Systems	Ms. Sunita Lama
29	Data Structures	Ms. Sunita Lama
30	Calculus I	Ms. Sunita Lama
31	Network Security	Ms. Sunita Lama
32	Ethical Hacking	Ms. Sunita Lama
33	Business Communication	Ms. Sunita Lama
34	Web Technologies	Ms. Sunita Lama
35	Operating Systems	Ms. Sunita Lama
36	Cloud Computing	Ms. Sunita Lama

Figure 42: Executing SQL join operation using cross join

9.2 Aggregate Functions

The following aggregate functions summarize and analyze data stored in the database.

- COUNT: Counts the total number of records.
- SUM: Calculates the total of numeric values.
- AVG: Calculates the average value of a column.
- MAX/MIN: Returns the highest and lowest values from a column.



The screenshot shows a SQL query in the 'Query Editor' window. The query is:

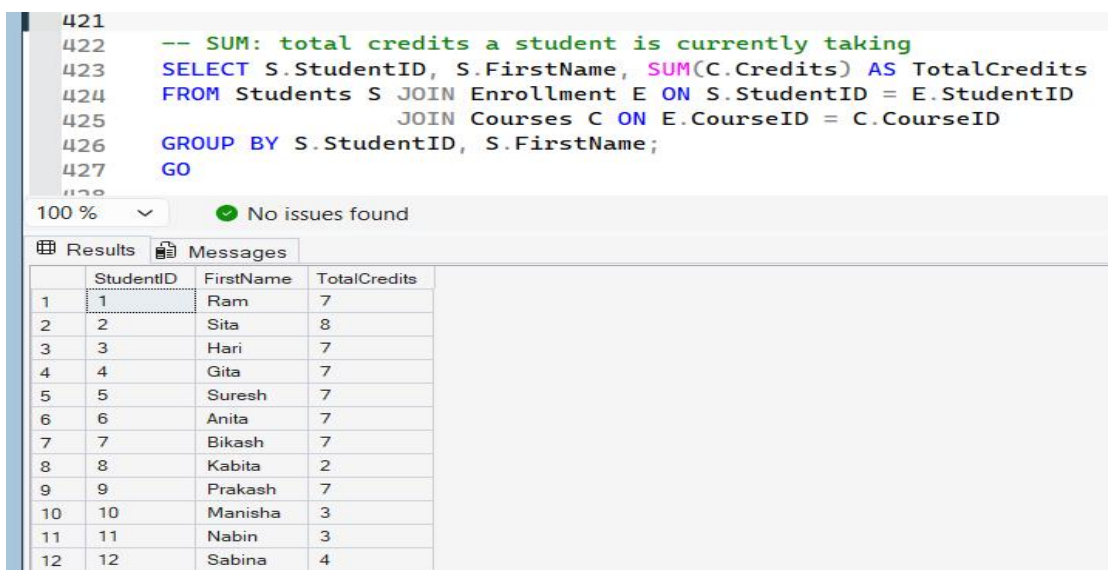

```
-- COUNT: students enrolled per course
SELECT C.CourseName, COUNT(E.StudentID) AS TotalStudents
FROM Courses C LEFT JOIN Enrollment E ON C.CourseID = E.CourseID
GROUP BY C.CourseName ORDER BY TotalStudents DESC;
GO
```

 The 'Results' pane shows the following data:

	CourseName	TotalStudents
1	Database Management Systems	4
2	Ethical Hacking	3
3	Network Security	3
4	Data Structures	3
5	Calculus I	2
6	Operating Systems	2
7	Web Technologies	2
8	Business Communication	1
9	Cloud Computing	0

 The status bar indicates 'No issues found'.

Figure 43: Executing aggregate function queries using COUNT



The screenshot shows a SQL query in the 'Query Editor' window. The query is:


```
-- SUM: total credits a student is currently taking
SELECT S.StudentID, S.FirstName, SUM(C.Credits) AS TotalCredits
FROM Students S JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Courses C ON E.CourseID = C.CourseID
GROUP BY S.StudentID, S.FirstName;
GO
```

 The 'Results' pane shows the following data:

	StudentID	FirstName	TotalCredits
1	1	Ram	7
2	2	Sita	8
3	3	Hari	7
4	4	Gita	7
5	5	Suresh	7
6	6	Anita	7
7	7	Bikash	7
8	8	Kabita	2
9	9	Prakash	7
10	10	Manisha	3
11	11	Nabin	3
12	12	Sabina	4

 The status bar indicates 'No issues found'.

Figure 44: Executing aggregate function queries using SUM

```

433 -- AVG: average grade points (numeric mapping) per course
434 SELECT C.CourseName, AVG(
435     CASE E.Grade WHEN 'A' THEN 4.0 WHEN 'A-' THEN 3.7 WHEN 'B+' THEN 3.3
436           WHEN 'B' THEN 3.0 WHEN 'B-' THEN 2.7 WHEN 'C+' THEN 2.3
437           WHEN 'C' THEN 2.0 WHEN 'D' THEN 1.0 ELSE 0 END) AS AvgGradePoint
438 FROM Enrollment E JOIN Courses C ON E.CourseID = C.CourseID
439 GROUP BY C.CourseName;
440 GO
441

```

100 % No issues found Ln: 431, C

	CourseName	AvgGradePoint
1	Business Communication	1.650000
2	Calculus I	3.350000
3	Data Structures	3.200000
4	Database Management Systems	3.600000
5	Ethical Hacking	3.800000
6	Network Security	3.666666
7	Operating Systems	3.150000
8	Web Technologies	3.650000

Figure 45: Executing aggregate function queries using AVG

```

442 -- MAX / MIN: highest and lowest credit courses and grade
443 SELECT MAX(Credits) AS HighestCredits, MIN(Credits) AS LowestCredits FROM Courses;
444 GO
445
446 SELECT C.CourseName,
447     MAX(CASE E.Grade WHEN 'A' THEN 4.0 WHEN 'A-' THEN 3.7 WHEN 'B+' THEN 3.3
448           WHEN 'B' THEN 3.0 WHEN 'B-' THEN 2.7 WHEN 'C+' THEN 2.3
449           WHEN 'C' THEN 2.0 WHEN 'D' THEN 1.0 ELSE 0 END) AS HighestGradePoint,
450     MIN(CASE E.Grade WHEN 'A' THEN 4.0 WHEN 'A-' THEN 3.7 WHEN 'B+' THEN 3.3
451           WHEN 'B' THEN 3.0 WHEN 'B-' THEN 2.7 WHEN 'C+' THEN 2.3
452           WHEN 'C' THEN 2.0 WHEN 'D' THEN 1.0 ELSE 0 END) AS LowestGradePoint
453 FROM Enrollment E
454 JOIN Courses C ON E.CourseID = C.CourseID
455 GROUP BY C.CourseName;
456 GO
457

```

100 % No issues found Ln: 437, C

	HighestCredits	LowestCredits
1	4	2

	CourseName	HighestGradePoint	LowestGradePoint
1	Business Communication	3.3	0.0
2	Calculus I	3.7	3.0
3	Data Structures	3.3	3.0
4	Database Management Systems	4.0	3.0
5	Ethical Hacking	4.0	3.7
6	Network Security	4.0	3.0
7	Operating Systems	3.3	3.0
8	Web Technologies	4.0	3.3

Figure 46: Executing aggregate function queries using MAX and MIN

9.3 Grouping & Filtering

The following queries organize records into groups and filter grouped results based on specified conditions.

GROUP BY: Groups records with similar values into summary results.

HAVING: Filters grouped records based on aggregate conditions.

```

465 -- GROUP BY: number of students per course
466 SELECT CourseID, COUNT(StudentID) AS TotalStudents FROM Enrollment GROUP BY CourseID;
467 GO
468
469 -- HAVING: only courses with more than 1 enrolled student
470 SELECT C.CourseName,
471        AVG(CASE E.Grade WHEN 'A' THEN 4.0 WHEN 'A-' THEN 3.7 WHEN 'B+' THEN 3.3
472                      WHEN 'B' THEN 3.5 WHEN 'B-' THEN 2.7 WHEN 'C+' THEN 2.3
473                      WHEN 'C' THEN 2.0 WHEN 'D' THEN 1.0 ELSE 0 END) AS AvgGradePoint
474 FROM Enrollment E
475 JOIN Courses C ON E.CourseID = C.CourseID
476 GROUP BY C.CourseName
477 HAVING AVG(CASE E.Grade WHEN 'A' THEN 4.0 WHEN 'A-' THEN 3.7 WHEN 'B+' THEN 3.3
478                WHEN 'B' THEN 3.5 WHEN 'B-' THEN 2.7 WHEN 'C+' THEN 2.3
479                WHEN 'C' THEN 2.0 WHEN 'D' THEN 1.0 ELSE 0 END) < 3.5;
480 GO
481

```

100 % No issues found

Results Messages

	CourseID	TotalStudents
1	1	4
2	2	3
3	3	1
4	4	3
5	5	3
6	6	2
7	7	2
8	8	2

	CourseName	AvgGradePoint
1	Business Communication	1.650000
2	Data Structures	3.366666
3	Operating Systems	3.400000

Figure 47: Executing GROUP BY and HAVING queries.

9.4 Subquery

A Sub-query is a query nested inside another SQL query. It is used to retrieve data based on the results of an inner query.

```

453
454 -- 6.9 Subquery: students who scored an 'A' in any course
455 v SELECT FirstName, LastName FROM Students
456    WHERE StudentID IN (SELECT StudentID FROM Enrollment WHERE Grade = 'A');
457 GO

```

100 % No issues found

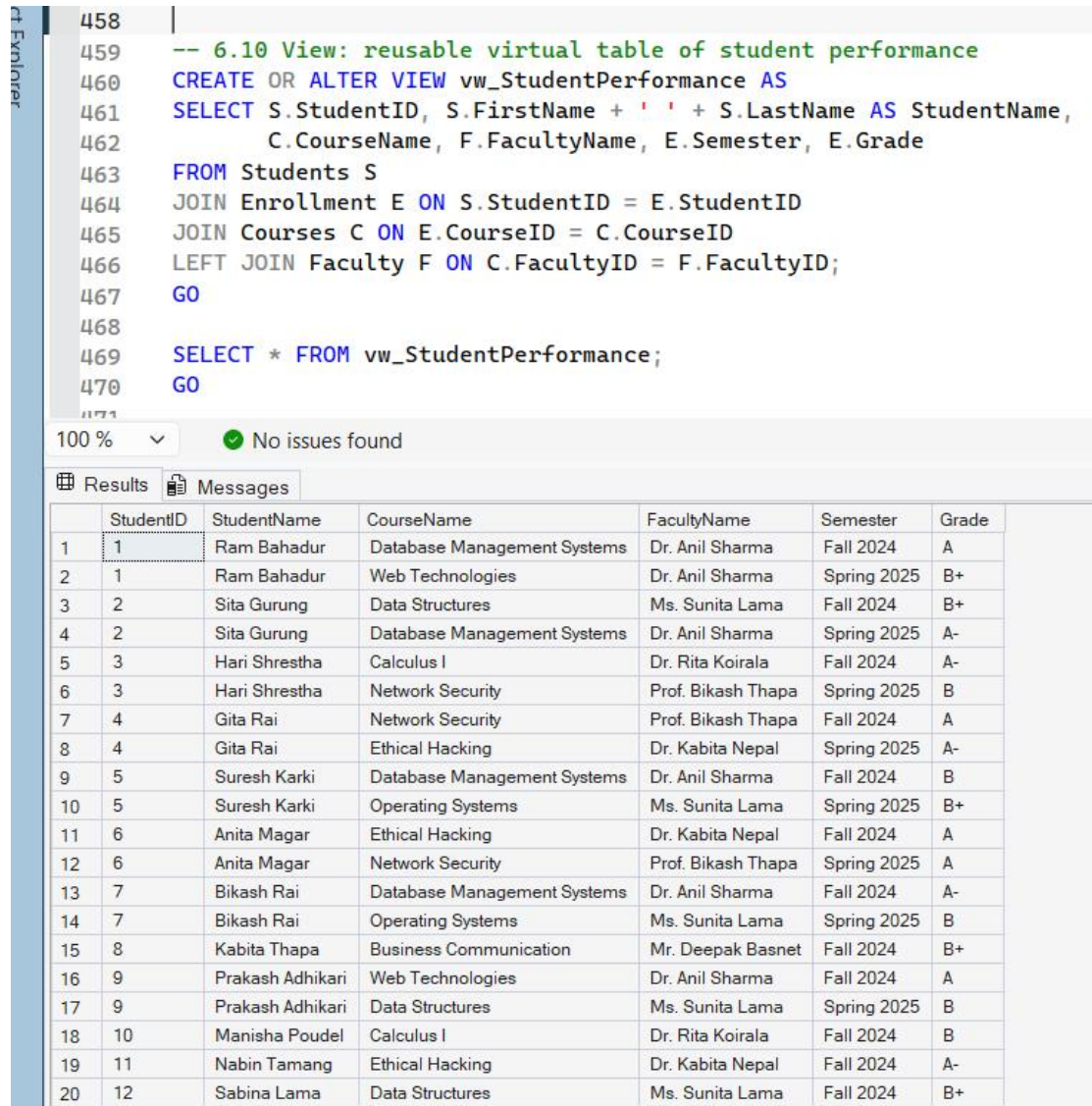
Results Messages

	FirstName	LastName
1	Ram	Bahadur
2	Gita	Rai
3	Anita	Magar
4	Prakash	Adhikari

Figure 48: Executing a sub-query to retrieve student records.

9.5 View

A View is a virtual table created from one or more database tables. It simplifies complex queries by presenting data in a reusable format.



The screenshot displays a SQL script in the 'Script' tab of SQL Server Enterprise Manager. The script creates a view named 'vw_StudentPerformance' by joining the 'Students', 'Enrollment', 'Courses', and 'Faculty' tables. Below the script, the 'Results' tab shows the output of the 'SELECT * FROM vw_StudentPerformance;' query, displaying a table with 7 columns: StudentID, StudentName, CourseName, FacultyName, Semester, and Grade. The table contains 20 rows of data.

```

458 |
459 | -- 6.10 View: reusable virtual table of student performance
460 | CREATE OR ALTER VIEW vw_StudentPerformance AS
461 | SELECT S.StudentID, S.FirstName + ' ' + S.LastName AS StudentName,
462 |        C.CourseName, F.FacultyName, E.Semester, E.Grade
463 | FROM Students S
464 | JOIN Enrollment E ON S.StudentID = E.StudentID
465 | JOIN Courses C ON E.CourseID = C.CourseID
466 | LEFT JOIN Faculty F ON C.FacultyID = F.FacultyID;
467 | GO
468 |
469 | SELECT * FROM vw_StudentPerformance;
470 | GO
  
```

	StudentID	StudentName	CourseName	FacultyName	Semester	Grade
1	1	Ram Bahadur	Database Management Systems	Dr. Anil Sharma	Fall 2024	A
2	1	Ram Bahadur	Web Technologies	Dr. Anil Sharma	Spring 2025	B+
3	2	Sita Gurung	Data Structures	Ms. Sunita Lama	Fall 2024	B+
4	2	Sita Gurung	Database Management Systems	Dr. Anil Sharma	Spring 2025	A-
5	3	Hari Shrestha	Calculus I	Dr. Rita Koirala	Fall 2024	A-
6	3	Hari Shrestha	Network Security	Prof. Bikash Thapa	Spring 2025	B
7	4	Gita Rai	Network Security	Prof. Bikash Thapa	Fall 2024	A
8	4	Gita Rai	Ethical Hacking	Dr. Kabita Nepal	Spring 2025	A-
9	5	Suresh Karki	Database Management Systems	Dr. Anil Sharma	Fall 2024	B
10	5	Suresh Karki	Operating Systems	Ms. Sunita Lama	Spring 2025	B+
11	6	Anita Magar	Ethical Hacking	Dr. Kabita Nepal	Fall 2024	A
12	6	Anita Magar	Network Security	Prof. Bikash Thapa	Spring 2025	A
13	7	Bikash Rai	Database Management Systems	Dr. Anil Sharma	Fall 2024	A-
14	7	Bikash Rai	Operating Systems	Ms. Sunita Lama	Spring 2025	B
15	8	Kabita Thapa	Business Communication	Mr. Deepak Basnet	Fall 2024	B+
16	9	Prakash Adhikari	Web Technologies	Dr. Anil Sharma	Fall 2024	A
17	9	Prakash Adhikari	Data Structures	Ms. Sunita Lama	Spring 2025	B
18	10	Manisha Poudel	Calculus I	Dr. Rita Koirala	Fall 2024	B
19	11	Nabin Tamang	Ethical Hacking	Dr. Kabita Nepal	Fall 2024	A-
20	12	Sabina Lama	Data Structures	Ms. Sunita Lama	Fall 2024	B+

Figure 49: Creating and retrieving data using a SQL view

9.6 Stored Procedure

A Stored Procedure is a precompiled collection of SQL statements that performs a specific task. It improves code reusability, consistency, and database performance.


```

471 |
472 -- 6.11 Stored Procedure: enroll a student in a course
473 CREATE OR ALTER PROCEDURE sp_EnrollStudent
474     @StudentID INT, @CourseID INT, @Semester VARCHAR(20)
475 AS
476 BEGIN
477     SET NOCOUNT ON;
478     IF NOT EXISTS (SELECT 1 FROM Enrollment WHERE StudentID = @StudentID AND CourseID = @CourseID AND Semester = @Semester)
479     BEGIN
480         INSERT INTO Enrollment (StudentID, CourseID, Semester) VALUES (@StudentID, @CourseID, @Semester);
481         PRINT 'Enrollment successful.';
482     END
483     ELSE
484         PRINT 'Student already enrolled in this course for this semester.';
485     END
486     GO
487
488 EXEC sp_EnrollStudent @StudentID = 13, @CourseID = 6, @Semester = 'Fall 2025';
489 GO
490

```

100 % No issues found

Messages

Enrollment successful.

Completion time: 2026-07-22T07:49:09.6723002+05:45

Figure 50: Creating and executing a stored procedure.

9.7 Trigger

A Trigger is a database object that automatically executes when a specified event, such as INSERT, UPDATE, or DELETE, occurs on a table. It helps maintain data integrity and automate database operations.

SQL Script

```

CREATE TABLE GradeChangeLog(
    LogID    INT IDENTITY(1,1) PRIMARY KEY,
    EnrollmentID INT,
    OldGrade  CHAR(2),
    NewGrade  CHAR(2),
    ChangedOn  DATETIME DEFAULT GETDATE());GO
CREATE OR ALTER TRIGGER trg_GradeChangeON EnrollmentAFTER
UPDATEASBEGIN
    SET NOCOUNT ON;
    IF UPDATE(Grade)

```

```
BEGIN
    INSERT INTO GradeChangeLog (EnrollmentID, OldGrade, NewGrade)
    SELECT d.EnrollmentID, d.Grade, i.Grade
    FROM deleted d JOIN inserted i ON d.EnrollmentID = i.EnrollmentID
    WHERE ISNULL(d.Grade,"") <> ISNULL(i.Grade,"");
ENDENDGO
UPDATE Enrollment SET Grade = 'A' WHERE EnrollmentID = 9;SELECT * FROM
GradeChangeLog;
GO
```

9.8 Index

An Index improves the performance of database queries by enabling faster retrieval of records from frequently searched columns.

SQL Script

```
CREATE NONCLUSTERED INDEX IX_Students_LastNames ON
Students(LastName);
CREATE NONCLUSTERED INDEX IX_Enrollments_CourseID ON
Enrollment(CourseID);
GO
```

10. CONCLUSION

The project was able to successfully design and develop a Student Information Management System using Microsoft SQL Server. In the course of the project, important database concepts like ER modeling, normalization, table creation, CRUD operations, SQL queries, relational algebra, transaction management, and advanced database concepts like views, stored procedure, trigger, and index were successfully utilized. Normalization helped reduce the redundancy of the data and improve the data consistency. Primary key, foreign key, and constraints have been used to make sure that the database is kept consistent. The implementation of SQL queries and database objects gave a better understanding on the design, management, and optimization of the relational databases. Data retrieval, transaction handling, and optimization techniques like indexing were also considered in the project. This project helped increase knowledge and skills about database management.

11. REFERENCES

- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387.
- Connolly, T., & Begg, C. (2015). *Database systems: A practical approach to design, implementation, and management* (6th ed.). Pearson.
- Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of database systems* (7th ed.). Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill Education.
- Microsoft. (2024a). CREATE TABLE (Transact-SQL). Microsoft Learn. <https://learn.microsoft.com/en-us/sql/t-sql/statements/create-table-transact-sql>
- Microsoft. (2024b). Transactions (Transact-SQL). Microsoft Learn. <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/transactions-transact-sql>

12. APPENDIX

Results		Messages	
	DepartmentID	DepartmentName	HOD
1	1	Computer Science	Dr. Anil Sharma
2	2	Mathematics	Dr. Rita Koirala
3	3	Cybersecurity	Prof. Bikash Thapa
4	4	Business Studies	Dr. Meena Adhikari

	FacultyID	FacultyName	Email	Phone	DepartmentID	AdminID	JoiningDate
1	1	Dr. Anil Sharma	anil.sharma@college.edu	9800000001	1	1	2006-01-09
2	2	Dr. Rita Koirala	rita.koirala@college.edu	9800000002	2	1	2004-09-23
3	3	Prof. Bikash Thapa	bikash.thapa@college.edu	9800000003	3	2	2005-06-29
4	4	Ms. Sunita Lama	sunita.lama@college.edu	9800000004	1	1	2012-03-15
5	5	Mr. Deepak Basnet	deepak.basnet@college.edu	9800000005	4	2	2015-08-01
6	6	Dr. Kabita Nepal	kabita.nepal@college.edu	9800000006	3	2	2010-11-20

	CourseID	CourseName	Credits	DepartmentID	FacultyID
1	1	Database Management Systems	4	1	1
2	2	Data Structures	4	1	4
3	3	Calculus I	3	2	2
4	4	Network Security	4	3	3
5	5	Ethical Hacking	3	3	6
6	6	Business Communication	2	4	5
7	7	Web Technologies	3	1	1
8	8	Operating Systems	4	1	4

	StudentID	FirstName	LastName	Gender	DOB	Email	Phone	Address	EnrollmentYear	DepartmentID	AdminID
1	1	Ram	Bahadur	M	2003-05-12	ram.bahadur@student.edu	9811111111	Kathmandu	2022	1	1
2	2	Sita	Gurung	F	2003-08-20	sita.gurung@student.edu	9822222222	Lalitpur	2022	1	1
3	3	Hari	Shrestha	M	2002-11-02	hari.shrestha@student.edu	9833333333	Bhaktapur	2021	2	2
4	4	Gita	Rai	F	2004-01-15	gita.rai@student.edu	9844444444	Kathmandu	2023	3	2
5	5	Suresh	Karki	M	2003-03-30	suresh.karki@student.edu	9855555555	Pokhara	2022	1	1
6	6	Anita	Magar	F	2004-06-18	anita.magar@student.edu	9866666666	Kathmandu	2023	3	2
7	7	Bikash	Rai	M	2004-02-20	bikash.rai@student.edu	9877777777	Kathmandu	2023	1	1
8	8	Kabita	Thapa	F	2003-09-09	kabita.thapa@student.edu	9888888888	Chitwan	2022	4	2
9	9	Prakash	Adhikari	M	2002-12-25	prakash.adhikari@stude...	9899999999	Kathmandu	2021	1	1
10	10	Manisha	Poudel	F	2004-04-04	manisha.poudel@studen...	9801111111	Lalitpur	2023	2	1
11	11	Nabin	Tamang	M	2003-07-07	nabin.tamang@student.e...	9802222222	Bhaktapur	2022	3	2
12	12	Sabina	Lama	F	2004-10-10	sabina.lama@student.edu	9803333333	Kathmandu	2023	1	1
13	13	Rajesh	KC	M	2002-05-05	rajesh.kc@student.edu	9804444444	Pokhara	2021	4	2
14	14	Puja	Basnet	F	2003-01-01	puja.basnet@student.edu	9805555555	Kathmandu	2022	2	1
15	15	Deepak	Khadka	M	2004-03-03	deepak.khadka@student...	9806666666	Lalitpur	2023	3	2

Results		Messages				
	EnrollmentID	StudentID	CourseID	Semester	Grade	EnrollmentDate
1	1	1	1	Fall 2024	A	2026-07-21
2	2	1	7	Spring 2025	B+	2026-07-21
3	3	2	2	Fall 2024	B+	2026-07-21
4	4	2	1	Spring 2025	A-	2026-07-21
5	5	3	3	Fall 2024	A-	2026-07-21
6	6	3	4	Spring 2025	B	2026-07-21
7	7	4	4	Fall 2024	A	2026-07-21
8	8	4	5	Spring 2025	A-	2026-07-21
9	9	5	1	Fall 2024	B	2026-07-21
10	10	5	8	Spring 2025	B+	2026-07-21
11	11	6	5	Fall 2024	A	2026-07-21
12	12	6	4	Spring 2025	A	2026-07-21
13	13	7	1	Fall 2024	A-	2026-07-21
14	14	7	8	Spring 2025	B	2026-07-21
15	15	8	6	Fall 2024	B+	2026-07-21
16	16	9	7	Fall 2024	A	2026-07-21
17	17	9	2	Spring 2025	B	2026-07-21
18	18	10	3	Fall 2024	B	2026-07-21
19	19	11	5	Fall 2024	A-	2026-07-21
20	20	12	2	Fall 2024	B+	2026-07-21